

Deploy a .NET 8 App to Azure Kubernetes Service (AKS) – Tutorial Guide

Kosisochukwu Ugochukwu

A beginner-friendly guide to deploying a .NET app to the cloud using Docker and Kubernetes.

Deploying applications to the cloud can seem daunting, especially if you are new to containers and Kubernetes. This guide is designed for beginners and walks you step by step through building a simple .NET 8 Web API, packaging it in a Docker container, and deploying it to Azure Kubernetes Service (AKS). By the end, you will also have automated deployments via GitHub Actions, so any code push instantly updates your app in the cloud.

Think of it as a hands-on way to learn modern cloud deployment, without getting lost in complex setups. By the time you finish, you will not only understand the tools but also gain confidence in deploying real-world apps to the cloud.

What You will Learn

By the end of this tutorial, you will know how to:

Build a .NET 8 Web API

Containerize your app with Docker

Deploy containers to Kubernetes in the cloud

Set up automated deployments with GitHub Actions

What You will be able Build :

Simple Weather API – Returns fake weather data (just like a real weather app)

Containerized App – Runs in Docker containers

Cloud Deployment – Hosted on Azure Kubernetes Service (AKS)

Automatic Updates – Push code to GitHub → auto-deploys to Azure

Required Tools to Set Up Your Computer

Install the following:

Visual Studio Code – code.visualstudio.com

.NET 9 SDK – dotnet.microsoft.com

Docker Desktop – docker.com/products/docker-desktop

Start Docker Desktop after installing

Azure CLI – docs.microsoft.com/cli/azure/install-azure-cli

kubectl (Kubernetes CLI) – kubernetes.io/docs/tasks/tools/

GitHub CLI – cli.github.com

Git – git-scm.com

Required Accounts

GitHub Account – github.com

Azure Account – azure.microsoft.com/free

Test Your Setup

Run these commands in your terminal (Command Prompt on Windows, Terminal on Mac/Linux) to verify:

```
dotnet --version
docker --version
az --version
kubectl version --client
gh --version
git --version
```

❏ ❏ ❏ ❏

You should see version numbers for all tools. If anything fails, reinstall that tool.

Step 1: Create the .NET Application

Let's start by building a simple .NET 8 Web API that we will later containerize and deploy to Azure.

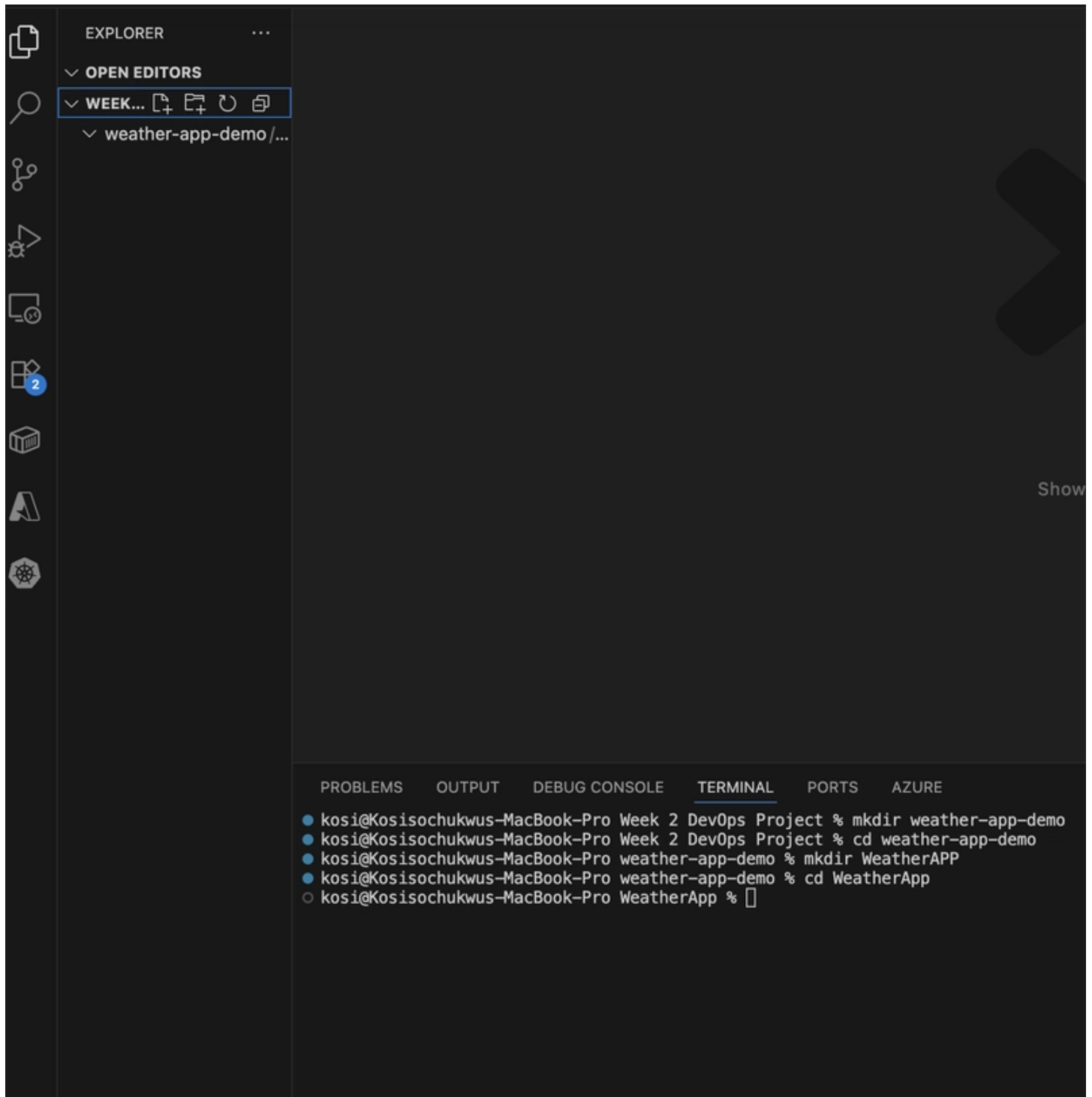
1.1 Set Up Project Structure

Create a new folder for your app:

```
mkdir weather-app-demo
```

```
cd weather-app-demo
mkdir WeatherApp
cd WeatherApp
```

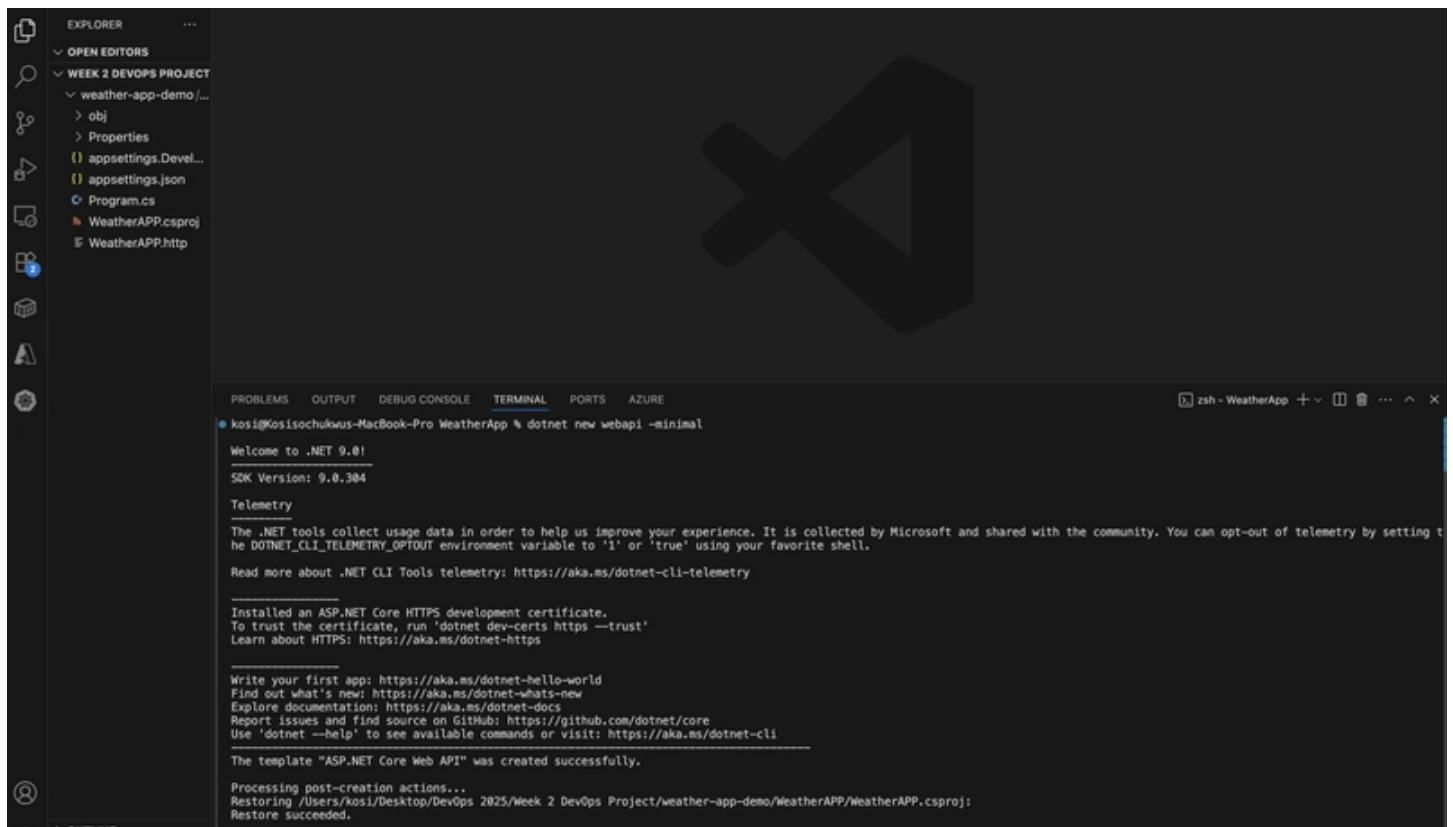
❏ ❏ ❏ ❏



1.2 Initialize a .NET Project

Generate a minimal Web API template:

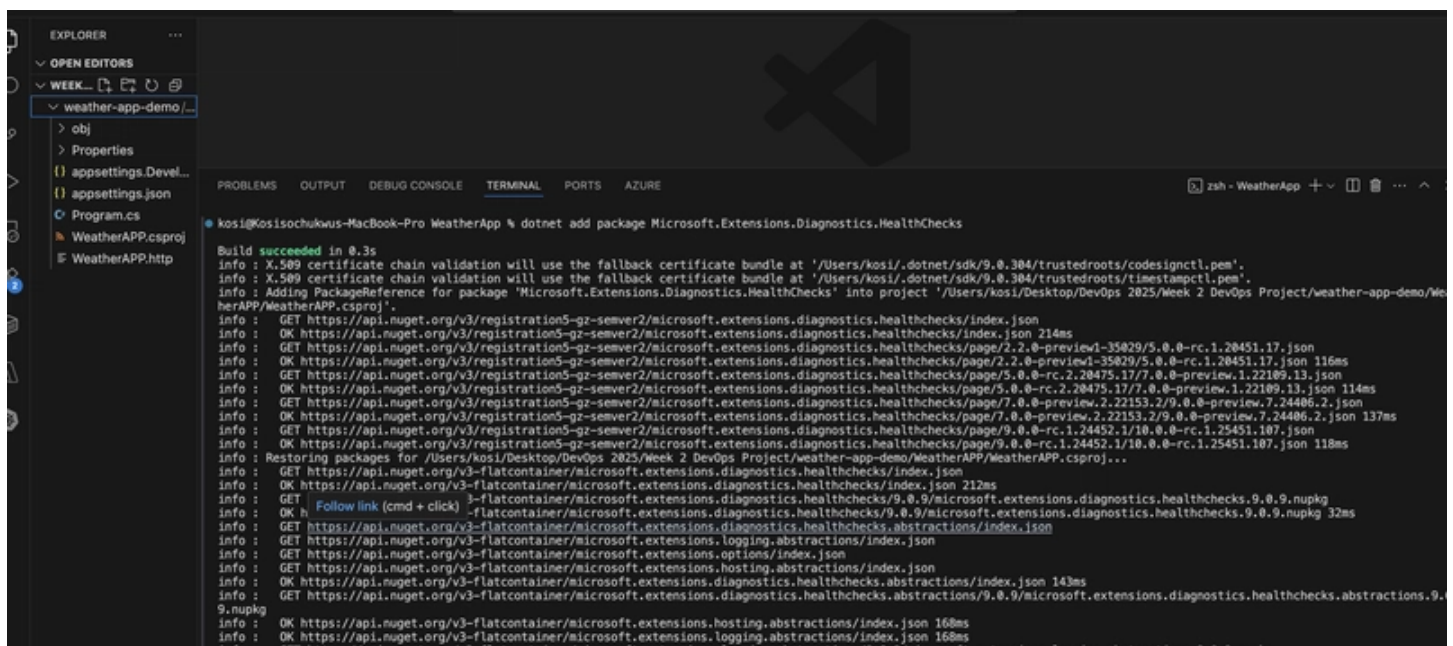
dotnet new webapi -minimal



1.3 Add Dependencies

Install the required packages:

```
dotnet add package Microsoft.Extensions.Diagnostics.HealthChecks
dotnet add package Swashbuckle.AspNetCore
```




```
Info : GET https://api.nuget.org/v3-flatcontainer/microsoft.extensions.logging.abstractions/9.0.9/microsoft.extensions.logging.abstractions.9.0.9.nupkg
Info : GET https://api.nuget.org/v3-flatcontainer/microsoft.extensions.hosting.abstractions/9.0.9/microsoft.extensions.hosting.abstractions.9.0.9.nupkg
Info : OK https://api.nuget.org/v3-flatcontainer/microsoft.extensions.diagnostics.healthchecks.abstractions/9.0.9/microsoft.extensions.diagnostics.healthchecks.abstractions.9.0.9.nupkg 24ms
Info : OK https://api.nuget.org/v3-flatcontainer/microsoft.extensions.options/index.json 176ms
Info : GET https://api.nuget.org/v3-flatcontainer/microsoft.extensions.options/9.0.9/microsoft.extensions.options.9.0.9.nupkg
Info : OK https://api.nuget.org/v3-flatcontainer/microsoft.extensions.logging.abstractions/9.0.9/microsoft.extensions.logging.abstractions.9.0.9.nupkg 13ms
Info : OK https://api.nuget.org/v3-flatcontainer/microsoft.extensions.hosting.abstractions/9.0.9/microsoft.extensions.hosting.abstractions.9.0.9.nupkg 13ms
Info : OK https://api.nuget.org/v3-flatcontainer/microsoft.extensions.options/9.0.9/microsoft.extensions.options.9.0.9.nupkg 13ms
Info : GET https://api.nuget.org/v3-flatcontainer/microsoft.extensions.dependencyinjection.abstractions/index.json
Info : GET https://api.nuget.org/v3-flatcontainer/microsoft.extensions.configuration.abstractions/index.json
Info : GET https://api.nuget.org/v3-flatcontainer/microsoft.extensions.diagnostics.abstractions/index.json
Info : GET https://api.nuget.org/v3-flatcontainer/microsoft.extensions.fileproviders.abstractions/index.json
Info : GET https://api.nuget.org/v3-flatcontainer/microsoft.extensions.primitives/index.json
Info : OK https://api.nuget.org/v3-flatcontainer/microsoft.extensions.configuration.abstractions/index.json 126ms
Info : GET https://api.nuget.org/v3-flatcontainer/microsoft.extensions.configuration.abstractions/9.0.9/microsoft.extensions.configuration.abstractions.9.0.9.nupkg
Info : OK https://api.nuget.org/v3-flatcontainer/microsoft.extensions.dependencyinjection.abstractions/index.json 138ms
Info : GET https://api.nuget.org/v3-flatcontainer/microsoft.extensions.dependencyinjection.abstractions/9.0.9/microsoft.extensions.dependencyinjection.abstractions.9.0.9.nupkg
```

```
EXPLORER
OPEN EDITORS
WEEK...
weather-app-demo/...
  obj
  Properties
  appsettings.Devel...
  - Desktop/DevOps 2025/Week 2 DevOps Project/weather-app-...
  demo/WeatherAPP/appsettings.Development.json
  WeatherAPP.csproj
  WeatherAPP.http

Build succeeded in 0.3s
Info : X.509 certificate chain validation will use the fallback certificate bundle at '/Users/kosli/.dotnet/sdk/9.0.304/trustedroots/codesignctl.pem'.
Info : X.509 certificate chain validation will use the fallback certificate bundle at '/Users/kosli/.dotnet/sdk/9.0.304/trustedroots/timestampctl.pem'.
Info : Adding PackageReference for package 'Swashbuckle.AspNetCore' into project '/Users/kosli/Desktop/DevOps 2025/Week 2 DevOps Project/weather-app-demo/WeatherAPP/WeatherAPP.csproj'.
Info : GET https://api.nuget.org/v3/registrations-gz-serve2/swashbuckle.aspnetcore/index.json
Info : OK https://api.nuget.org/v3/registrations-gz-serve2/swashbuckle.aspnetcore/index.json 218ms
Info : Restoring packages for /Users/kosli/Desktop/DevOps 2025/Week 2 DevOps Project/weather-app-demo/WeatherAPP/WeatherAPP.csproj...
Info : GET https://api.nuget.org/v3-flatcontainer/swashbuckle.aspnetcore/index.json
Info : OK https://api.nuget.org/v3-flatcontainer/swashbuckle.aspnetcore/index.json 178ms
Info : GET https://api.nuget.org/v3-flatcontainer/swashbuckle.aspnetcore/9.0.4/swashbuckle.aspnetcore.9.0.4.nupkg
Info : OK https://api.nuget.org/v3-flatcontainer/swashbuckle.aspnetcore/9.0.4/swashbuckle.aspnetcore.9.0.4.nupkg 38ms
Info : GET https://api.nuget.org/v3-flatcontainer/swashbuckle.aspnetcore.swagger/index.json
Info : GET https://api.nuget.org/v3-flatcontainer/swashbuckle.aspnetcore.swaggergen/index.json
Info : GET https://api.nuget.org/v3-flatcontainer/swashbuckle.aspnetcore.swaggerui/index.json
Info : GET https://api.nuget.org/v3-flatcontainer/swashbuckle.aspnetcore.apidescription.server/index.json
Info : OK https://api.nuget.org/v3-flatcontainer/swashbuckle.aspnetcore.swagger/index.json 133ms
Info : GET https://api.nuget.org/v3-flatcontainer/swashbuckle.aspnetcore.swagger/9.0.4/swashbuckle.aspnetcore.swagger.9.0.4.nupkg
Info : OK https://api.nuget.org/v3-flatcontainer/swashbuckle.aspnetcore.swaggergen/index.json 176ms
Info : GET https://api.nuget.org/v3-flatcontainer/swashbuckle.aspnetcore.swaggergen/9.0.4/swashbuckle.aspnetcore.swaggergen.9.0.4.nupkg
Info : GET https://api.nuget.org/v3-flatcontainer/swashbuckle.aspnetcore.swaggerui/index.json 179ms
Info : GET https://api.nuget.org/v3-flatcontainer/swashbuckle.aspnetcore.swaggerui/9.0.4/swashbuckle.aspnetcore.swaggerui.9.0.4.nupkg
Info : OK https://api.nuget.org/v3-flatcontainer/swashbuckle.aspnetcore.swagger/9.0.4/swashbuckle.aspnetcore.swagger.9.0.4.nupkg 51ms
Info : OK https://api.nuget.org/v3-flatcontainer/swashbuckle.aspnetcore.swaggergen/9.0.4/swashbuckle.aspnetcore.swaggergen.9.0.4.nupkg 17ms
Info : GET https://api.nuget.org/v3-flatcontainer/microsoft.openapi/index.json
Info : OK https://api.nuget.org/v3-flatcontainer/microsoft.openapi/index.json 197ms
Info : GET https://api.nuget.org/v3-flatcontainer/microsoft.extensions.apidescription.server/index.json 197ms
Info : OK https://api.nuget.org/v3-flatcontainer/microsoft.extensions.apidescription.server/9.0.0/microsoft.extensions.apidescription.server.9.0.0.nupkg 30ms
Info : OK https://api.nuget.org/v3-flatcontainer/microsoft.openapi/index.json 165ms
Info : GET https://api.nuget.org/v3-flatcontainer/microsoft.openapi/1.6.25/microsoft.openapi.1.6.25.nupkg
Info : OK https://api.nuget.org/v3-flatcontainer/microsoft.openapi/1.6.25/microsoft.openapi.1.6.25.nupkg 42ms
Info : Installed Swashbuckle.AspNetCore.Swagger 9.0.4 from https://api.nuget.org/v3/index.json to /Users/kosli/.nuget/packages/swashbuckle.aspnetcore.swagger/9.0.4 with content hash syUB4Eg30fMhH8eDh66erdsY0hp82InXbrQdW3p0fQ8BtePCTetkLlNanovHfX48aLZ8E6gQ0f8ZKQ==.
Info : Installed Swashbuckle.AspNetCore 9.0.4 from https://api.nuget.org/v3/index.json to /Users/kosli/.nuget/packages/swashbuckle.aspnetcore/9.0.4 with content hash 4hghaogM5b7IvjJ8576G5xrsjX2pNvahl5NjZSLr2QCV8ZyI8wB89AXC1SGkxVt9/bbs51EXZr/nxj0=.
Info : Installed Swashbuckle.AspNetCore.SwaggerGen 9.0.4 from https://api.nuget.org/v3/index.json to /Users/kosli/.nuget/packages/swashbuckle.aspnetcore.swaggergen/9.0.4 with content hash GncIkaq7kapEckG5rVnK121CRDebyr14/753T/rQ007e0DIKxtjTQJZqaz0xaTXBDCD5fIm1YMQHfN5Q==.
Info : Installed Microsoft.OpenApi 1.6.25 from https://api.nuget.org/v3/index.json to /Users/kosli/.nuget/packages/microsoft.openapi/1.6.25 with content hash ZahSgNGtNW/NbJBYS/IYXpkLVeXl/AZfao6qpxv6A7U17Q7zfiNjkaQ1csV73Ukzx141l3dy0g0UKIM8Cwm=.
Info : Installed Swashbuckle.AspNetCore.SwaggerUI 9.0.4 from https://api.nuget.org/v3/index.json to /Users/kosli/.nuget/packages/swashbuckle.aspnetcore.swaggerui/9.0.4 with content hash 2up72DvZsHqHk7ZrDmupDufet18Wv2UxaAfmH8QMTovb7Z8u50QZa6208Tg8f/rQ2S6w4Ypfymd=.
Info : Installed Microsoft.Extensions.ApiDescription.Server 9.0.0 from https://api.nuget.org/v3/index.json to /Users/kosli/.nuget/packages/microsoft.extensions.apidescription.server/9.0.0 with content hash 1Kzf7pReY48VWk9W9/wwrKVku2AQjt+GveeKLpIEHAJ1x2Rsl5h4ZZYfny57Kt20B0PmsXNdi+wbC0l5wm=.
Info : CACHE https://api.nuget.org/v3/vulnerabilities/index.json
Info : CACHE https://api.nuget.org/v3-vulnerabilities/2025.09.06.03.24.34/vulnerability.base.json
Info : CACHE https://api.nuget.org/v3-vulnerabilities/2025.09.06.03.24.34/2025.09.09.15.24.42/vulnerability.update.json
Info : Package 'Swashbuckle.AspNetCore' is compatible with all the specified frameworks in project '/Users/kosli/Desktop/DevOps 2025/Week 2 DevOps Project/weather-app-demo/WeatherAPP/WeatherAPP.csproj'.
```

HealthChecks – adds a /health endpoint (important for Kubernetes).

Swashbuckle – provides Swagger UI for API docs.

1.4 Write the Application Code

Replace the contents of Program.cs with the following:

```
var builder = WebApplication.CreateBuilder(args);

// Add services to the container
builder.Services.AddHealthChecks();
builder.Services.AddEndpointsApiExplorer();
builder.Services.AddSwaggerGen();

// Configure Kestrel to listen on port 8080 (required for containers)
```

```
builder.WebHost.ConfigureKestrel(options =>
{
    options.ListenAnyIP(8080);
});

var app = builder.Build();

// Configure the HTTP request pipeline
if (app.Environment.IsDevelopment() || app.Environment.IsProduction())
{
    app.UseSwagger();
    app.UseSwaggerUI(c =>
    {
        c.SwaggerEndpoint("/swagger/v1/swagger.json", "Weather API v1");
        c.RoutePrefix = "swagger";
    });
}

// Define API endpoints
app.MapGet("/", () => new
{
    Message = "Welcome to the Weather App!",
    Version = "1.0.0",
    Environment = app.Environment.EnvironmentName,
    Timestamp = DateTime.UtcNow
})
.WithName("GetWelcome")
.WithTags("General");

app.MapGet("/weather", () =>
{
    var summaries = new[] {
        "Freezing", "Bracing", "Chilly", "Cool", "Mild",
        "Warm", "Balmy", "Hot", "Sweltering", "Scorching"
    };

    var forecast = Enumerable.Range(1, 5).Select(index => new
```

```

{
    Date = DateOnly.FromDateTime(DateTime.Now.AddDays(index)),
    TemperatureC = Random.Shared.Next(-20, 55),
    TemperatureF = 0,
    Summary = summaries[Random.Shared.Next(summaries.Length)]
})
.Select(temp => new
{
    temp.Date,
    temp.TemperatureC,
    TemperatureF = 32 + (int)(temp.TemperatureC / 0.5556),
    temp.Summary
});

return forecast;
})
.WithName("GetWeatherForecast")
.WithTags("Weather");

// Health check endpoint (required for Kubernetes)
app.MapHealthChecks("/health")
.WithTags("Health");

app.Run();

```

❏ ❏ ❏ ❏

```

EXPLORER
...
OPEN EDITORS
  Program.cs we...
WEEK 2 DEVOPS PROJECT
  weather-app-demo/...
    bin
    obj
    Properties
    appsettings.Devel...
    appsettings.json
    Program.cs
    WeatherAPP.csproj
    WeatherAPP.http
    Week 2 DevOps Proj...

Program.cs
weather-app-demo > WeatherAPP > Program.cs > Program > <top-level-statements-entry-point>
1  var builder = WebApplication.CreateBuilder(args);
2
3  // Add services to the container
4  builder.Services.AddHealthChecks();
5  builder.Services.AddEndpointsApiExplorer();
6  builder.Services.AddSwaggerGen();
7
8  // Configure Kestrel to listen on port 8080 (required for containers)
9  builder.WebHost.ConfigureKestrel(options =>
10 {
11     options.ListenAnyIP(8080);
12 });
13
14 var app = builder.Build();
15
16 // Configure the HTTP request pipeline
17 if (app.Environment.IsDevelopment() || app.Environment.IsProduction())
18 {
19     app.UseSwagger();
20     app.UseSwaggerUI(c =>

```

```

21     {
22         c.SwaggerEndpoint("/swagger/v1/swagger.json", "Weather API v1");
23         c.RoutePrefix = "swagger";
24     });
25 }
26
27 // Define API endpoints
28 app.MapGet("/", () => new
29 {
30     Message = "Welcome to the Weather App!",
31     Version = "1.0.0",
32     Environment = app.Environment.EnvironmentName,
33     Timestamp = DateTime.UtcNow
34 })
35     .WithName("GetWelcome")
36     .WithTags("General");
37
38 app.MapGet("/weather", () =>
39 {
40     var summaries = new[] {
41         "Freezing", "Bracing", "Chilly", "Cool", "Mild",

```

1.5 Run and Test Locally

Start the application:

dotnet run

```

PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS  AZURE
kosi@Kosisochukwu-MacBook-Pro WeatherApp % dotnet run
Using launch settings from /Users/kosi/Desktop/DevOps 2025/Week 2 DevOps Project/weather-app-demo/WeatherAPP/Properties/launchSettings.json...
Building...
warn: Microsoft.AspNetCore.Server.Kestrel[0]
      Overriding address(es) 'http://localhost:5085'. Binding to endpoints defined via IConfiguration and/or UseKestrel() instead.
info: Microsoft.Hosting.Lifetime[14]
      Now listening on: http://[::]:8080
info: Microsoft.Hosting.Lifetime[0]
      Application started. Press Ctrl+C to shut down.
info: Microsoft.Hosting.Lifetime[0]
      Hosting environment: Development
info: Microsoft.Hosting.Lifetime[0]
      Content root path: /Users/kosi/Desktop/DevOps 2025/Week 2 DevOps Project/weather-app-demo/WeatherAPP

```

Now, test the endpoints in your browser:

http://localhost:8080/ → Welcome message

```

Pretty print ☐
{"message":"Welcome to the Weather App!","version":"1.0.0","environment":"Development","timestamp":"2025-09-09T18:41:51.83651Z"}

```

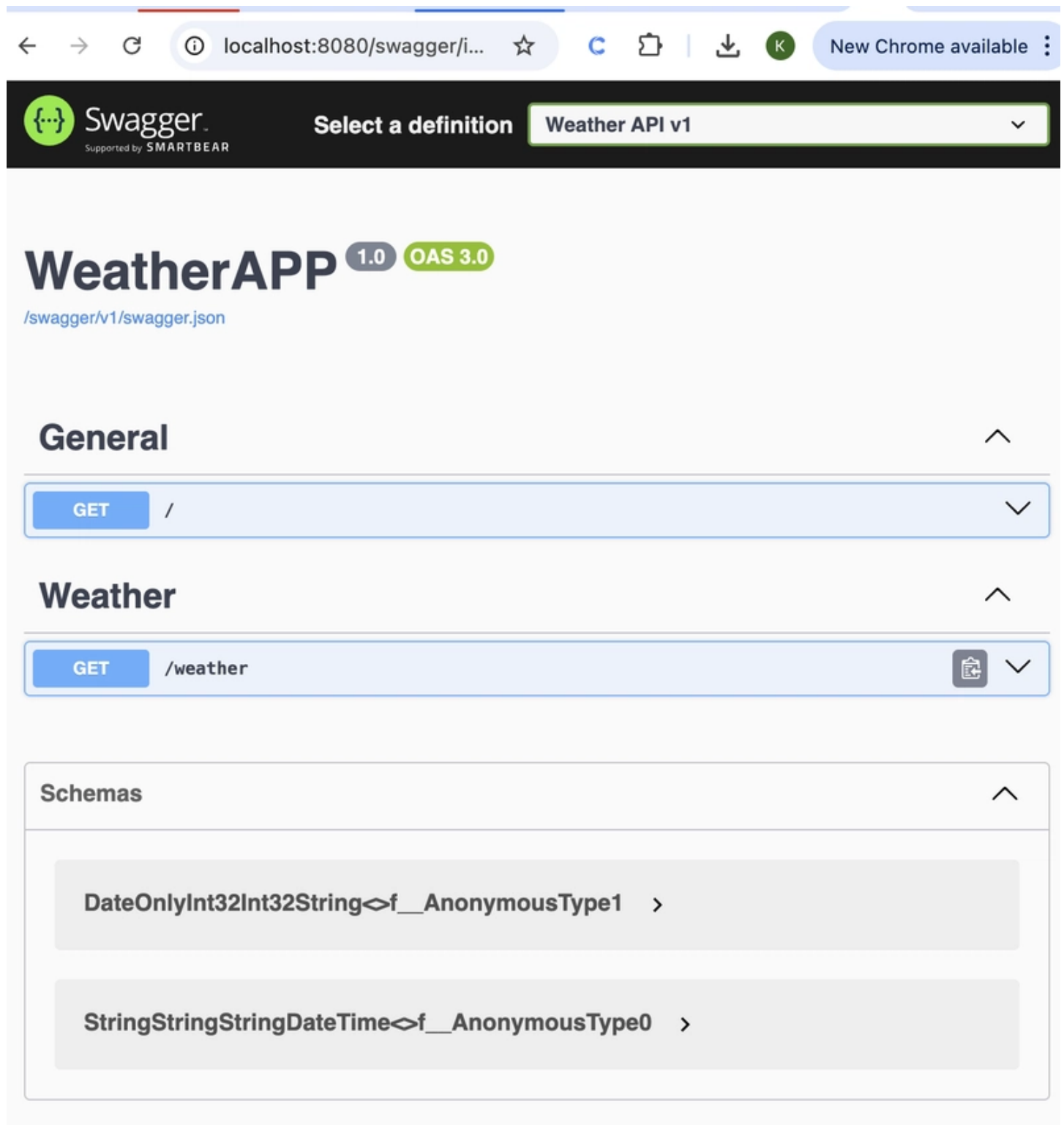
http://localhost:8080/weather → Weather forecast

```

Pretty print ☐
[{"date":"2025-09-10","temperatureC":8,"temperatureF":46,"summary":"Scorching"}, {"date":"2025-09-11","temperatureC":22,"temperatureF":71,"summary":"Freezing"}, {"date":"2025-09-12","temperatureC":53,"temperatureF":127,"summary":"Freezing"}, {"date":"2025-09-13","temperatureC":43,"temperatureF":109,"summary":"Sweltering"}, {"date":"2025-09-14","temperatureC":23,"temperatureF":73,"summary":"Mild"}]

```

http://localhost:8080/swagger → API docs



The screenshot shows the Swagger UI for the Weather API v1. The browser address bar displays 'localhost:8080/swagger/i...'. The Swagger logo is in the top left, and a dropdown menu shows 'Weather API v1'. The main title is 'WeatherAPP 1.0 OAS 3.0' with a link to '/swagger/v1/swagger.json'. There are three expandable sections: 'General' (containing a GET endpoint '/'), 'Weather' (containing a GET endpoint '/weather'), and 'Schemas' (containing two schema definitions: 'DateOnlyInt32Int32String' and 'StringStringStringDateTime').

Swagger
Supported by SMARTBEAR

Select a definition Weather API v1

WeatherAPP 1.0 OAS 3.0

/swagger/v1/swagger.json

General

GET /

Weather

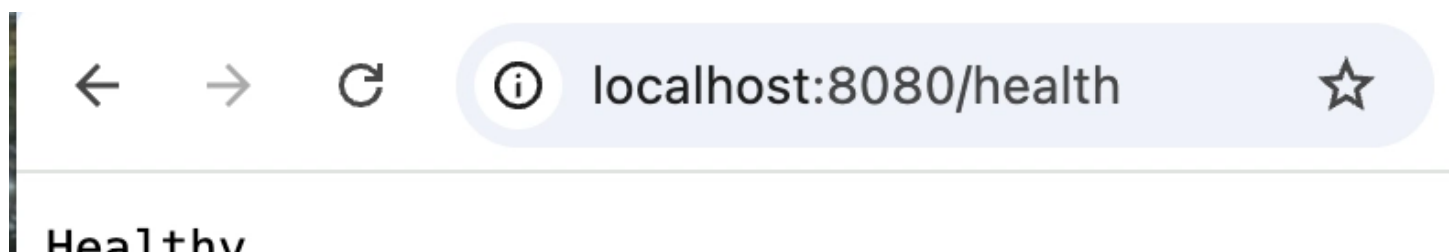
GET /weather

Schemas

DateOnlyInt32Int32String <f__AnonymousType1 >

StringStringStringDateTime <f__AnonymousType0 >

http://localhost:8080/health → Health check endpoint



The screenshot shows the browser address bar with the URL 'localhost:8080/health'. The page title 'Healthv' is partially visible on the left.

Healthv

localhost:8080/health

neat city

Press Ctrl + C to stop the app.

```
Content root path: /Users/kosi/Desktop/DevOps 2025/Week 2 DevOps Project/weather-app-demo/WeatherAPP
^Cinfo: Microsoft.Hosting.Lifetime[0]
Application is shutting down...
kosi@Kosisochukwus-MacBook-Pro WeatherApp %
```

At this point, you have a working .NET API that's container-ready. Next, we will Dockerize the app so it can run inside a container.

Step 2: Containerize Your Application

Now that the app runs locally, let's package it into a Docker container so it can run consistently anywhere.

2.1 Create a Dockerfile

Inside the WeatherApp folder, create a file named Dockerfile (no extension) and add the following:

```
# Multi-stage build for optimized image size

# Runtime stage
FROM mcr.microsoft.com/dotnet/aspnet:9.0 AS base
WORKDIR /app
EXPOSE 8080
ENV ASPNETCORE_URLS=http://+:8080
ENV ASPNETCORE_ENVIRONMENT=Production

# Build stage
FROM mcr.microsoft.com/dotnet/sdk:9.0 AS build
ARG BUILD_CONFIGURATION=Release
WORKDIR /src
```



```

# Copy project file and restore dependencies
COPY ["WeatherApp/WeatherApp.csproj", "."]
RUN dotnet restore "WeatherApp.csproj"

# Copy source code and build
COPY . .
WORKDIR /src
COPY WeatherApp/ ./WeatherApp/
WORKDIR /src/WeatherApp
RUN dotnet restore "WeatherApp.csproj"
RUN dotnet build "WeatherApp.csproj" -c $BUILD_CONFIGURATION -o /app/build

# Publish stage
FROM build AS publish
ARG BUILD_CONFIGURATION=Release
RUN dotnet publish "WeatherApp.csproj" -c $BUILD_CONFIGURATION -o /app/publish /
p:UseAppHost=false

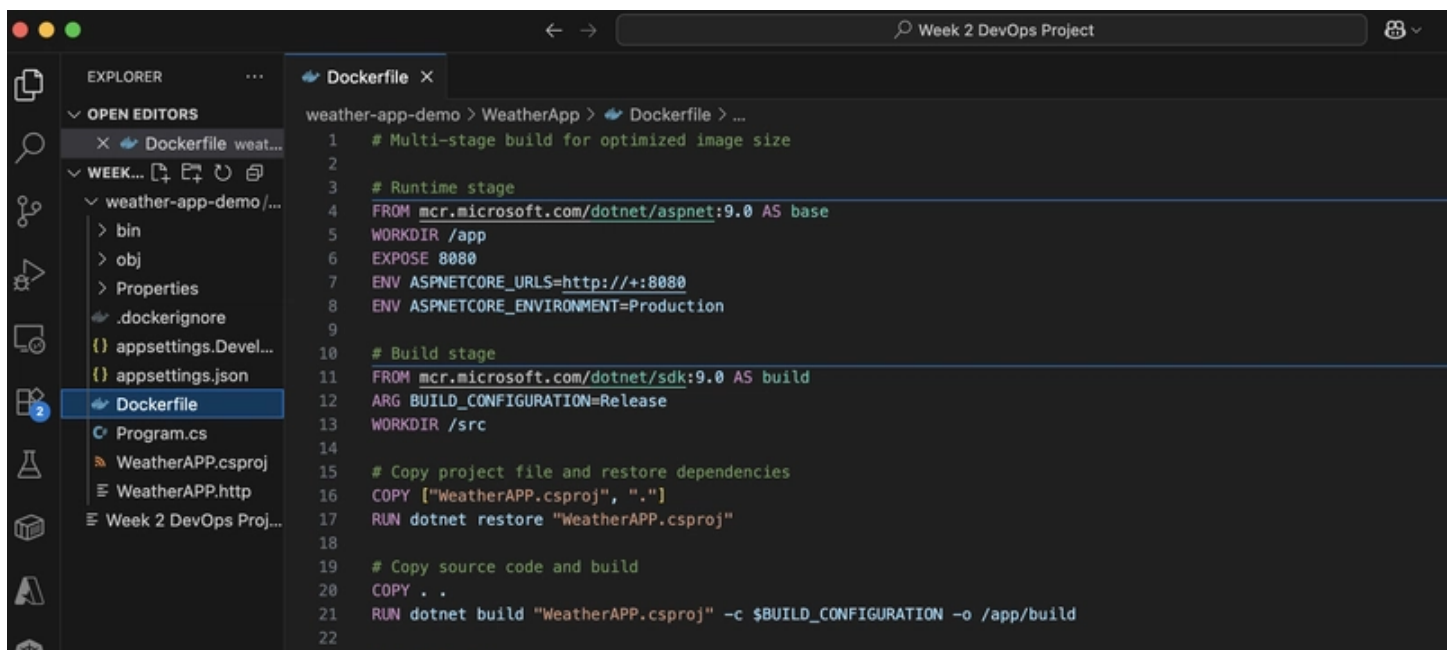
# Final stage
FROM base AS final
WORKDIR /app
COPY --from=publish /app/publish .
ENTRYPOINT ["dotnet", "WeatherApp.dll"]

```

```

[] [] [] []
[] [] [] []

```



```

weather-app-demo > WeatherApp > Dockerfile > ...
1  # Multi-stage build for optimized image size
2
3  # Runtime stage
4  FROM mcr.microsoft.com/dotnet/aspnet:9.0 AS base
5  WORKDIR /app
6  EXPOSE 8080
7  ENV ASPNETCORE_URLS=http://+:8080
8  ENV ASPNETCORE_ENVIRONMENT=Production
9
10 # Build stage
11 FROM mcr.microsoft.com/dotnet/sdk:9.0 AS build
12 ARG BUILD_CONFIGURATION=Release
13 WORKDIR /src
14
15 # Copy project file and restore dependencies
16 COPY ["WeatherAPP.csproj", "."]
17 RUN dotnet restore "WeatherAPP.csproj"
18
19 # Copy source code and build
20 COPY . .
21 RUN dotnet build "WeatherAPP.csproj" -c $BUILD_CONFIGURATION -o /app/build
22

```

```
23 # Publish stage
24 FROM build AS publish
25 ARG BUILD_CONFIGURATION=Release
26 RUN dotnet publish "WeatherAPP.csproj" -c $BUILD_CONFIGURATION -o /app/publish /p:UseAppHost=false
27
28 # Final stage
29 FROM base AS final
```

This uses a multi-stage build to keep the final image lightweight:

Build stage compiles your app.

Publish stage outputs optimized binaries.

Final stage only contains what's needed to run.

2.2 Create a .dockerignore File

Create a .dockerignore file in the root of your project to prevent unnecessary files from being copied into the Docker image:

```
# Build outputs
bin/
obj/
out/

# IDE files
.vs/
.vscode/
*.user
*.suo

# OS files
.DS_Store
Thumbs.db

# Git
.git/
.gitignore

# Documentation
README.md
*.md

# Docker
```



```
Dockerfile*  
.dockerignore
```

```
# Logs  
*.log  
logs/
```

```
❏ ❏ ❏ ❏  
❏ ❏ ❏ ❏
```

This keeps your Docker image clean and small.

2.3 Build and Test the Container

Build the Docker Image

```
docker build -t weather-app:local .
```

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS  AZURE  
● kosi@Mac WeatherApp % docker build -t weather-app:local .  
[+] Building 10.5s (17/17) FINISHED  
=> [internal] load build definition from Dockerfile  
=> => transferring dockerfile: 876B  
=> [internal] load metadata for mcr.microsoft.com/dotnet/sdk:9.0  
=> [internal] load metadata for mcr.microsoft.com/dotnet/aspnet:9.0  
=> [internal] load .dockerignore  
=> => transferring context: 257B  
=> [build 1/6] FROM mcr.microsoft.com/dotnet/sdk:9.0@sha256:bb42ae2c058609d1746baf24fe6864ecab0686711dfca1f4b7a99e367ab17162  
=> => resolve mcr.microsoft.com/dotnet/sdk:9.0@sha256:bb42ae2c058609d1746baf24fe6864ecab0686711dfca1f4b7a99e367ab17162  
=> => sha256:a92950d7d7b03013b9d4cbb9c147385b162ab1ef0449d4904b464b4d1a46c0c0 17.49MB / 17.49MB  
=> => sha256:8074155faa864cef21c7b3741886140f546be478443bd8c46c00b0e92668b59a 2.73kB / 2.73kB  
=> => sha256:761d7fc060657915cd0f92838a7ab9f747779db22d22fd98e8bfa1bbd54a78db 173.09MB / 173.09MB  
=> => sha256:13902f43502420af4e20a5ce998be9cf4734e933ebebcbf72b91e6e5337aefcd 31.27MB / 31.27MB  
=> => extracting sha256:13902f43502420af4e20a5ce998be9cf4734e933ebebcbf72b91e6e5337aefcd  
=> => extracting sha256:761d7fc060657915cd0f92838a7ab9f747779db22d22fd98e8bfa1bbd54a78db  
=> => extracting sha256:8074155faa864cef21c7b3741886140f546be478443bd8c46c00b0e92668b59a  
=> => extracting sha256:a92950d7d7b03013b9d4cbb9c147385b162ab1ef0449d4904b464b4d1a46c0c0  
=> [base 1/2] FROM mcr.microsoft.com/dotnet/aspnet:9.0@sha256:1af4114db9ba87542a3f23dbb5cd9072cad7fcc8505f6e9131d1feb580286a6f  
=> => resolve mcr.microsoft.com/dotnet/aspnet:9.0@sha256:1af4114db9ba87542a3f23dbb5cd9072cad7fcc8505f6e9131d1feb580286a6f  
=> => sha256:6ebc9803b7838a96fac0b9d6f94161760f8b9ba0113397926c377018fb9757f4 165B / 165B  
=> => sha256:9ac552aa62458db3faebb01b25ed9686e701d246b36979312e6fc28de139dbc1 3.33kB / 3.33kB  
=> => sha256:41d064a923ee055c5ee33e083d832cbc241f40ce7326c22e8d422dd89b792f4e 10.92MB / 10.92MB  
=> => sha256:769a0d861171b28cbc50b88cb124f67a10d790be762b18b7eedb9794578d5c0b 32.85MB / 32.85MB  
=> => sha256:2b3fbfbf7c03c29967edbb69ad72e8d9919b244082446b5fb57bc00c2686a7f 18.53MB / 18.53MB  
=> => sha256:0878ecc8b0afd0d835641c015541aacd4780ec19e5565a3e1a5af3f77d208d42 28.10MB / 28.10MB  
=> => extracting sha256:0878ecc8b0afd0d835641c015541aacd4780ec19e5565a3e1a5af3f77d208d42  
=> => extracting sha256:2b3fbfbf7c03c29967edbb69ad72e8d9919b244082446b5fb57bc00c2686a7f  
=> => extracting sha256:9ac552aa62458db3faebb01b25ed9686e701d246b36979312e6fc28de139dbc1  
=> => extracting sha256:769a0d861171b28cbc50b88cb124f67a10d790be762b18b7eedb9794578d5c0b  
=> => extracting sha256:6ebc9803b7838a96fac0b9d6f94161760f8b9ba0113397926c377018fb9757f4  
=> => extracting sha256:41d064a923ee055c5ee33e083d832cbc241f40ce7326c22e8d422dd89b792f4e  
=> [internal] load build context
```

Run the Container

```
docker run -d -p 8080:8080 --name weather-test weather-app:local
```

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS  AZURE  
● kosi@Mac WeatherApp % docker run -d -p 8080:8080 --name weather-test weather-app:local  
aec9b494c73bf05441ed22f289b63f0c9d45dad19947f5153dece52dad823d1e  
○ kosi@Mac WeatherApp %
```

Test the App Inside Docker

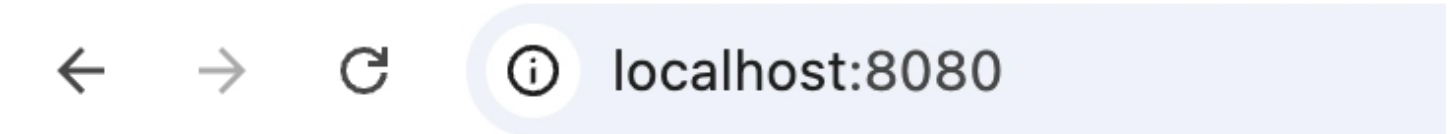
curl http://localhost:8080/

curl http://localhost:8080/weather

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS AZURE
kosi@Mac Week 2 DevOps Project % curl http://localhost:8080/
{"message":"Welcome to the Weather App!","version":"1.0.0","environment":"Production","timestamp":"2025-09-12T10:30:01.2592765Z"}
kosi@Mac Week 2 DevOps Project % curl http://localhost:8080/weather
[{"date":"2025-09-13","temperatureC":5,"temperatureF":40,"summary":"Balmy"}, {"date":"2025-09-14","temperatureC":14,"temperatureF":57,"summary":"Warm"}, {"date":"2025-09-15","temperatureC":42,"temperatureF":107,"summary":"Balmy"}, {"date":"2025-09-16","temperatureC":29,"temperatureF":84,"summary":"Warm"}, {"date":"2025-09-17","temperatureC":54,"temperatureF":129,"summary":"Hot"}]
```

Or just visit in your browser:

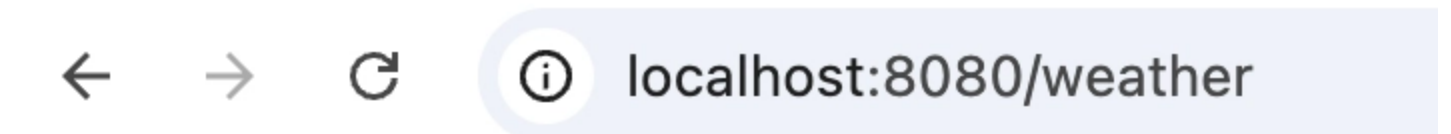
http://localhost:8080/



pretty print ☒

```
{
  "message": "Welcome to the Weather App!",
  "version": "1.0.0",
  "environment": "Production",
  "timestamp": "2025-09-12T12:34:32.788382Z"
}
```

<http://localhost:8080/weather>



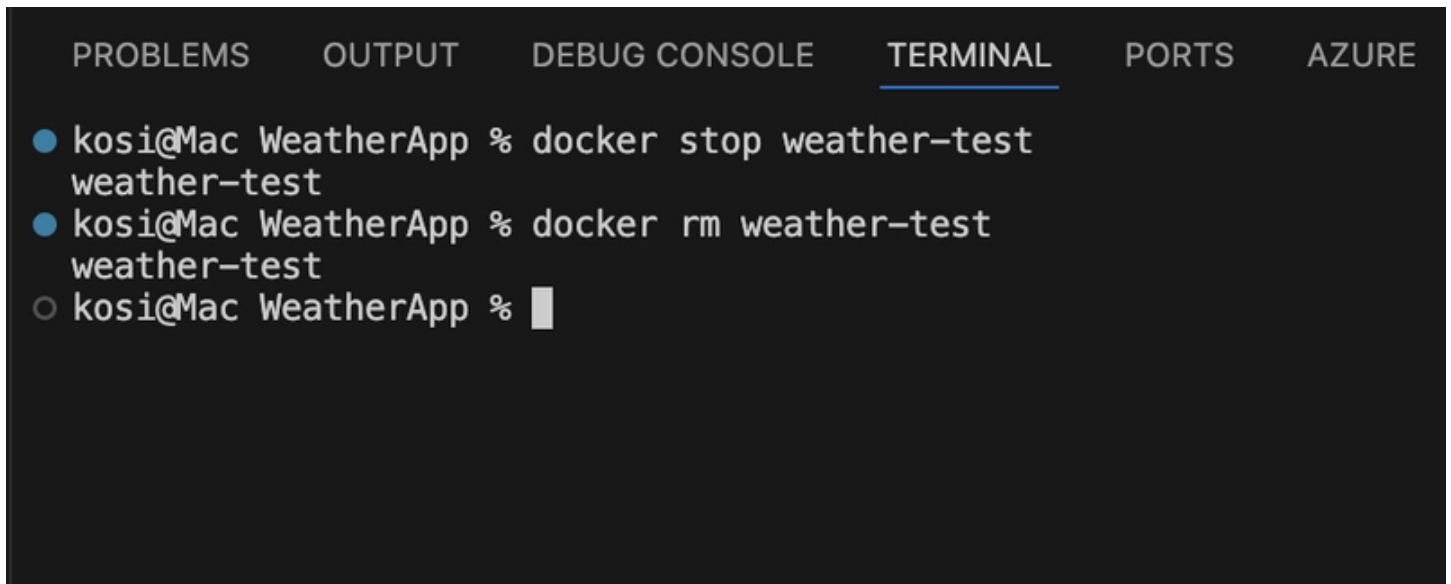
Pretty print ☒

```
[
  {
    "date": "2025-09-13",
    "temperatureC": 43,
    "temperatureF": 109,
    "summary": "Scorching"
  },
  {
    "date": "2025-09-14",
    "temperatureC": -14,
    "temperatureF": 7,
    "summary": "Cool"
  },
  {
    "date": "2025-09-15",
    "temperatureC": 14,
    "temperatureF": 57,
    "summary": "Warm"
  },
  {
    "date": "2025-09-16",
    "temperatureC": 36,
    "temperatureF": 96,
    "summary": "Freezing"
  },
  {
    "date": "2025-09-17",
    "temperatureC": 48,
    "temperatureF": 118,
    "summary": "Freezing"
  }
]
```

]

Stop & Remove the Container

```
docker stop weather-test  
docker rm weather-test
```

A screenshot of a terminal window with a dark background. At the top, there are tabs labeled 'PROBLEMS', 'OUTPUT', 'DEBUG CONSOLE', 'TERMINAL' (which is selected and underlined), 'PORTS', and 'AZURE'. The terminal shows three lines of text: a blue bullet point followed by 'kosi@Mac WeatherApp % docker stop weather-test', a second blue bullet point followed by 'kosi@Mac WeatherApp % docker rm weather-test', and a grey circle followed by 'kosi@Mac WeatherApp %' and a cursor. The first two lines have been executed, and the third is the current prompt.

```
● kosi@Mac WeatherApp % docker stop weather-test  
weather-test  
● kosi@Mac WeatherApp % docker rm weather-test  
weather-test  
○ kosi@Mac WeatherApp % █
```

At this point, your app is running inside a container. Next, we'll push the container to a registry and deploy it to Azure Kubernetes Service (AKS).

Step 3: Set Up Azure Infrastructure

Now that your app runs inside a container, it's time to deploy it to the cloud. We will use Azure Kubernetes Service (AKS), along with Azure Container Registry (ACR) to store and distribute your Docker images.

3.1 Authenticate with Azure

Log in to your Azure account:

```
az login
```

This opens your browser for sign-in.

3.2 Select Your Subscription

List your available subscriptions:

```
az account list --output table
```

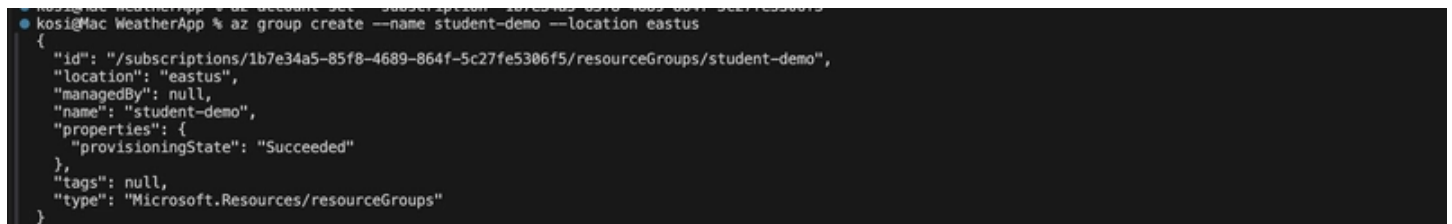
Set the subscription you want to use:

```
az account set --subscription "Your-Subscription-Name"
```

3.3 Create a Resource Group

A resource group is a logical container for all your Azure resources:

```
az group create --name student-demo --location eastus
```

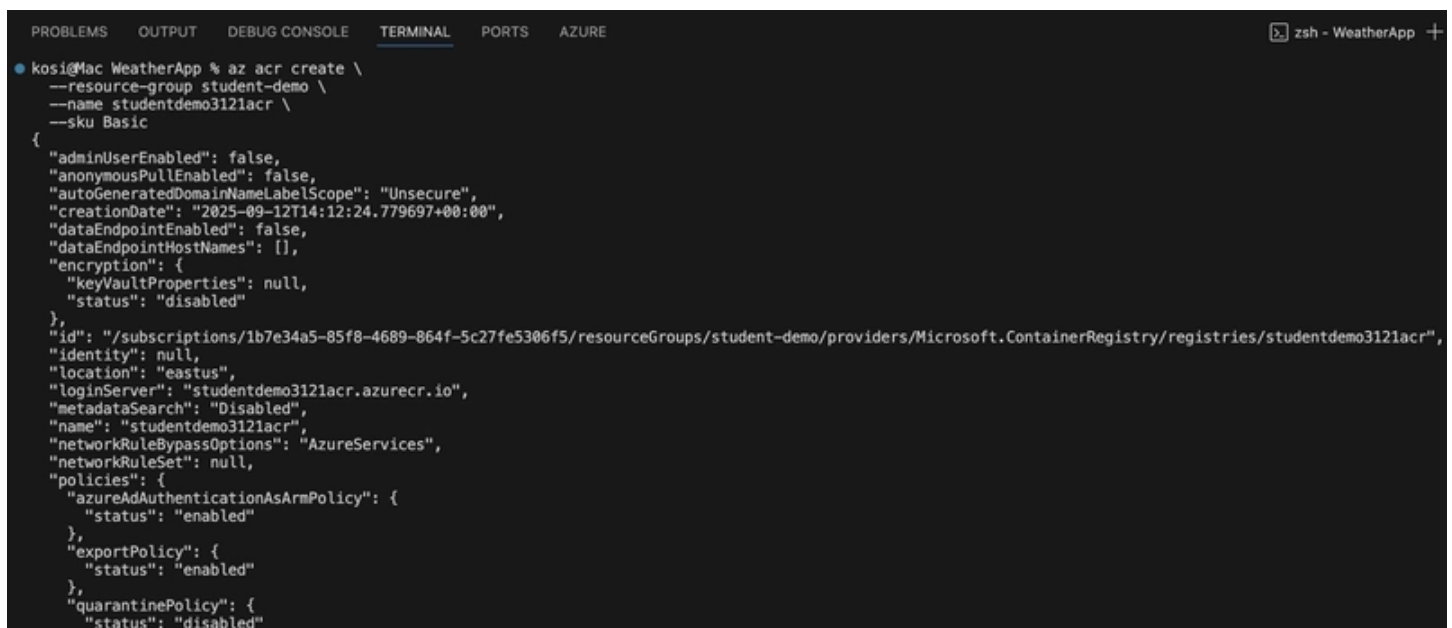


```
kosi@Mac WeatherApp % az group create --name student-demo --location eastus
{
  "id": "/subscriptions/1b7e34a5-85f8-4689-864f-5c27fe5306f5/resourceGroups/student-demo",
  "location": "eastus",
  "managedBy": null,
  "name": "student-demo",
  "properties": {
    "provisioningState": "Succeeded"
  },
  "tags": null,
  "type": "Microsoft.Resources/resourceGroups"
}
```

3.4 Create a Container Registry (ACR)

We will use ACR to store our Docker images. Make sure the name is unique (append initials/year if needed):

```
az acr create \
--resource-group student-demo \
--name studentdemo3121acr \
--sku Basic
```



```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS AZURE
kosi@Mac WeatherApp % az acr create \
--resource-group student-demo \
--name studentdemo3121acr \
--sku Basic
{
  "adminUserEnabled": false,
  "anonymousPullEnabled": false,
  "autoGeneratedDomainNameLabelScope": "Unsecure",
  "creationDate": "2025-09-12T14:12:24.779697+00:00",
  "dataEndpointEnabled": false,
  "dataEndpointHostNames": [],
  "encryption": {
    "keyVaultProperties": null,
    "status": "disabled"
  },
  "id": "/subscriptions/1b7e34a5-85f8-4689-864f-5c27fe5306f5/resourceGroups/student-demo/providers/Microsoft.ContainerRegistry/registries/studentdemo3121acr",
  "identity": null,
  "location": "eastus",
  "loginServer": "studentdemo3121acr.azurecr.io",
  "metadataSearch": "Disabled",
  "name": "studentdemo3121acr",
  "networkRuleBypassOptions": "AzureServices",
  "networkRuleSet": null,
  "policies": {
    "azureAdAuthenticationAsArmPolicy": {
      "status": "enabled"
    },
    "exportPolicy": {
      "status": "enabled"
    },
    "quarantinePolicy": {
      "status": "disabled"
    }
  }
}
```

Note if you have an error while creating RG for your container registry, use this code below to register

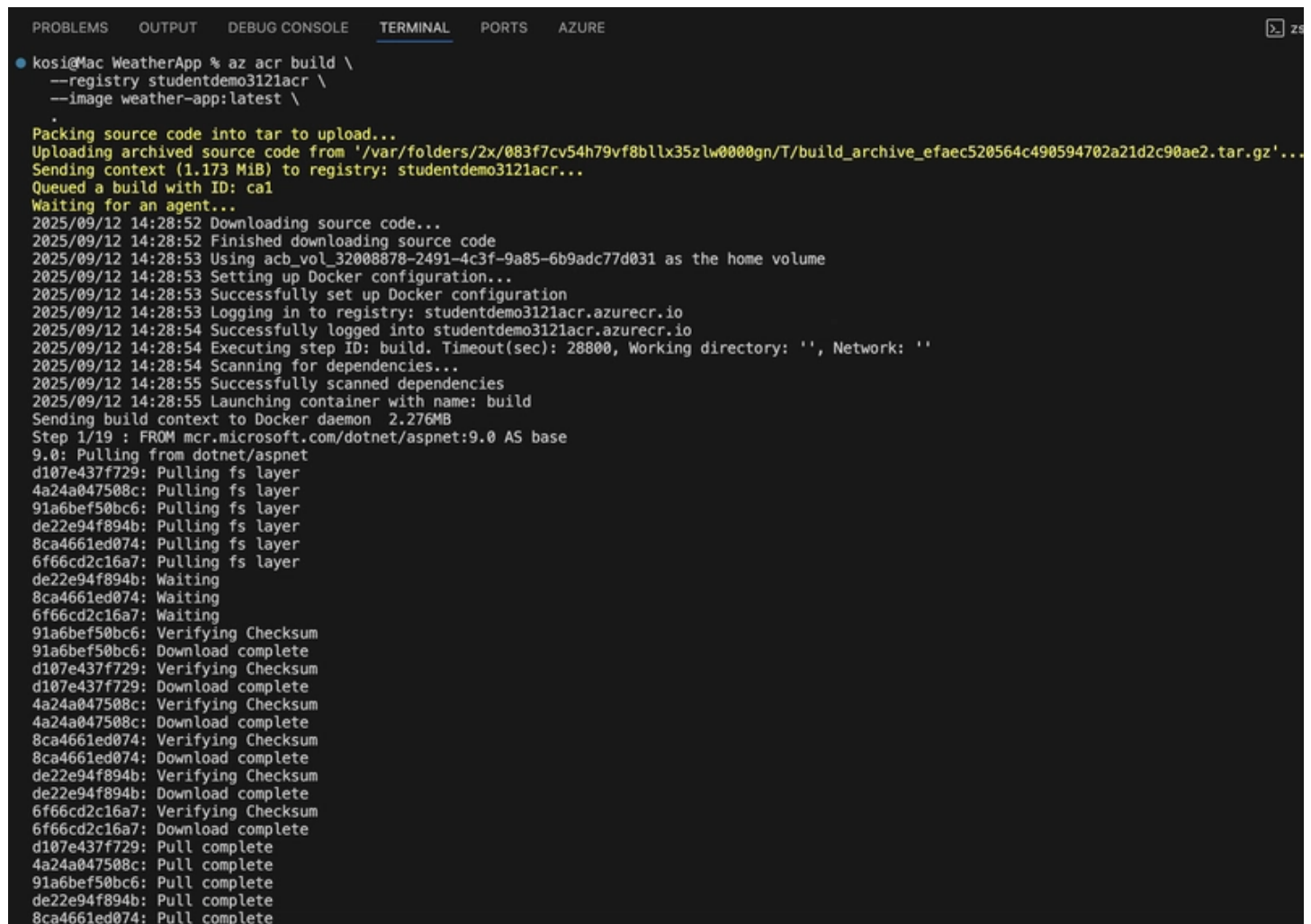
namespace on your container registry: `az provider register --namespace Microsoft.ContainerRegistry`

3.5 Build & Push the Image to ACR

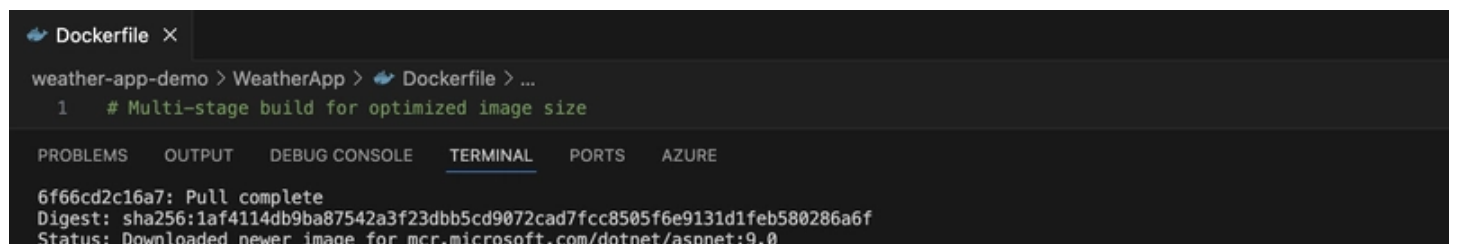
Instead of building locally and pushing manually, we will let Azure build and push the image for us using `az acr build`.

From the folder containing your Dockerfile:

```
az acr build \  
--registry studentdemo3121acr \  
--image weather-app:latest \  
.
```



```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS  AZURE  
● kosi@Mac WeatherApp % az acr build \  
  --registry studentdemo3121acr \  
  --image weather-app:latest \  
  .  
Packing source code into tar to upload...  
Uploading archived source code from '/var/folders/2x/083f7cv54h79vf8bllx35zlw0000gn/T/build_archive_efaec520564c490594702a21d2c90ae2.tar.gz'...  
Sending context (1.173 MiB) to registry: studentdemo3121acr...  
Queued a build with ID: cal  
Waiting for an agent...  
2025/09/12 14:28:52 Downloading source code...  
2025/09/12 14:28:52 Finished downloading source code  
2025/09/12 14:28:53 Using acb_vol_32008878-2491-4c3f-9a85-6b9adc77d031 as the home volume  
2025/09/12 14:28:53 Setting up Docker configuration...  
2025/09/12 14:28:53 Successfully set up Docker configuration  
2025/09/12 14:28:53 Logging in to registry: studentdemo3121acr.azurecr.io  
2025/09/12 14:28:54 Successfully logged into studentdemo3121acr.azurecr.io  
2025/09/12 14:28:54 Executing step ID: build. Timeout(sec): 28800, Working directory: '', Network: ''  
2025/09/12 14:28:54 Scanning for dependencies...  
2025/09/12 14:28:55 Successfully scanned dependencies  
2025/09/12 14:28:55 Launching container with name: build  
Sending build context to Docker daemon 2.276MB  
Step 1/19 : FROM mcr.microsoft.com/dotnet/aspnet:9.0 AS base  
9.0: Pulling from dotnet/aspnet  
d107e437f729: Pulling fs layer  
4a24a047508c: Pulling fs layer  
91a6bef50bc6: Pulling fs layer  
de22e94f894b: Pulling fs layer  
8ca4661ed074: Pulling fs layer  
6f66cd2c16a7: Pulling fs layer  
de22e94f894b: Waiting  
8ca4661ed074: Waiting  
6f66cd2c16a7: Waiting  
91a6bef50bc6: Verifying Checksum  
91a6bef50bc6: Download complete  
d107e437f729: Verifying Checksum  
d107e437f729: Download complete  
4a24a047508c: Verifying Checksum  
4a24a047508c: Download complete  
8ca4661ed074: Verifying Checksum  
8ca4661ed074: Download complete  
de22e94f894b: Verifying Checksum  
de22e94f894b: Download complete  
6f66cd2c16a7: Verifying Checksum  
6f66cd2c16a7: Download complete  
d107e437f729: Pull complete  
4a24a047508c: Pull complete  
91a6bef50bc6: Pull complete  
de22e94f894b: Pull complete  
8ca4661ed074: Pull complete
```



```
Dockerfile X  
weather-app-demo > WeatherApp > Dockerfile > ...  
1 # Multi-stage build for optimized image size  
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS  AZURE  
6f66cd2c16a7: Pull complete  
Digest: sha256:1af4114db9ba87542a3f23dbb5cd9072cad7fcc8505f6e9131d1feb580286a6f  
Status: Downloaded newer image for mcr.microsoft.com/dotnet/aspnet:9.0
```

```

--> 42df88dcd080
Step 2/19 : WORKDIR /app
--> Running in a9223cf2ea8f
Removing intermediate container a9223cf2ea8f
--> f7ec24200974
Step 3/19 : EXPOSE 8080
--> Running in 228e70095741
Removing intermediate container 228e70095741
--> b92fd3b259be
Step 4/19 : ENV ASPNETCORE_URLS=http://+:8080
--> Running in 1e17743a3e6b
Removing intermediate container 1e17743a3e6b
--> e3c251d73215
Step 5/19 : ENV ASPNETCORE_ENVIRONMENT=Production
--> Running in 0d645ce69ca8
Removing intermediate container 0d645ce69ca8
--> 37cdd6954400
Step 6/19 : FROM mcr.microsoft.com/dotnet/sdk:9.0 AS build
9.0: Pulling from dotnet/sdk
d107e437f729: Already exists
4a24a047508c: Already exists
91a6bef50bc6: Already exists
de22e94f894b: Already exists
8ca4661ed074: Already exists
6f66cd2c16a7: Already exists
42a538ee5b89: Pulling fs layer
68aa7387efe8: Pulling fs layer
ea29a56e41db: Pulling fs layer
70a8d67e460e: Pulling fs layer
70a8d67e460e: Waiting
ea29a56e41db: Download complete
42a538ee5b89: Verifying Checksum
42a538ee5b89: Download complete
70a8d67e460e: Verifying Checksum
70a8d67e460e: Download complete
68aa7387efe8: Verifying Checksum
68aa7387efe8: Download complete
42a538ee5b89: Pull complete
68aa7387efe8: Pull complete
ea29a56e41db: Pull complete
70a8d67e460e: Pull complete
Digest: sha256:bb42ae2c058609d1746baf24fe6864ecab0686711dfca1f4b7a99e367ab17162
Status: Downloaded newer image for mcr.microsoft.com/dotnet/sdk:9.0
--> 1aeff3957680
Step 7/19 : ARG BUILD_CONFIGURATION=Release
--> Running in 8756b1f0065b

```

Dockerfile X

weather-app-demo > WeatherApp > Dockerfile > ...

```
1 # Multi-stage build for optimized image size
```

PROBLEMS	OUTPUT	DEBUG CONSOLE	TERMINAL	PORTS	AZURE
			<pre> Step 7/19 : ARG BUILD_CONFIGURATION=Release --> Running in 8756b1f0065b Removing intermediate container 8756b1f0065b --> d4143e132aa8 Step 8/19 : WORKDIR /src --> Running in cfec14156300 Removing intermediate container cfec14156300 --> 67b1823df5d4 Step 9/19 : COPY ["WeatherAPP.csproj", "."] --> b5ee4b45fb48 Step 10/19 : RUN dotnet restore "WeatherAPP.csproj" --> Running in 4877d63810e9 Determining projects to restore... Restored /src/WeatherAPP.csproj (in 1.9 sec). Removing intermediate container 4877d63810e9 --> 92f4da912e0a Step 11/19 : COPY . . --> 345e03757e48 Step 12/19 : RUN dotnet build "WeatherAPP.csproj" -c \$BUILD_CONFIGURATION -o /app/build --> Running in de89ff0ef441 Determining projects to restore... Restored /src/WeatherAPP.csproj (in 250 ms). WeatherAPP -> /app/build/WeatherAPP.dll Build succeeded. 0 Warning(s) 0 Error(s) Time Elapsed 00:00:04.62 Removing intermediate container de89ff0ef441 --> 85c3720d48ac Step 13/19 : FROM build AS publish </pre>		


```

--> 85c3720d48ac
Step 14/19 : ARG BUILD_CONFIGURATION=Release
--> Running in 99ae39dc5c93
Removing intermediate container 99ae39dc5c93
--> 8a172099b0e3
Step 15/19 : RUN dotnet publish "WeatherAPP.csproj" -c $BUILD_CONFIGURATION -o /app/publish /p:UseAppHost=false
--> Running in e7dba66b0823
    Determining projects to restore...
    All projects are up-to-date for restore.
    WeatherAPP -> /src/bin/Release/net9.0/WeatherAPP.dll
    WeatherAPP -> /app/publish/
Removing intermediate container e7dba66b0823
--> 3f110a250ffc
Step 16/19 : FROM base AS final
--> 37cdd6954400
Step 17/19 : WORKDIR /app
--> Running in 4b646fd02473

```

Dockerfile X

weather-app-demo > WeatherApp > Dockerfile > ...

1 # Multi-stage build for optimized image size

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS AZURE

```

2025/09/12 14:29:32 Executing step ID: push. Timeout(sec): 3600, Working directory: '', Network: ''
2025/09/12 14:29:32 Pushing image: studentdemo3121acr.azurecr.io/weather-app:latest, attempt 1
The push refers to repository [studentdemo3121acr.azurecr.io/weather-app]
dd580f61dba3: Preparing
ab53bc7d5374: Preparing
9b4f978c8fc6: Preparing
4289c63563cf: Preparing
3a3dbab1a8ca: Preparing
c0691992e8c0: Preparing
6dfd5c4c2433: Preparing
36f5f951f60a: Preparing
c0691992e8c0: Waiting
6dfd5c4c2433: Waiting
36f5f951f60a: Waiting
ab53bc7d5374: Pushed
9b4f978c8fc6: Pushed
4289c63563cf: Pushed
c0691992e8c0: Pushed
dd580f61dba3: Pushed
3a3dbab1a8ca: Pushed
6dfd5c4c2433: Pushed
36f5f951f60a: Pushed
latest: digest: sha256:8c851e7980d33de792af7cd87e1f6de8a9d592fb803c5d478a7980b70ae76dc3 size: 1997
2025/09/12 14:29:41 Successfully pushed image: studentdemo3121acr.azurecr.io/weather-app:latest
2025/09/12 14:29:41 Step ID: build marked as successful (elapsed time in seconds: 37.329014)
2025/09/12 14:29:41 Populating digests for step ID: build...
2025/09/12 14:29:42 Successfully populated digests for step ID: build
2025/09/12 14:29:42 Step ID: push marked as successful (elapsed time in seconds: 9.242887)
2025/09/12 14:29:42 The following dependencies were found:
2025/09/12 14:29:42
- image:
  registry: studentdemo3121acr.azurecr.io
  repository: weather-app
  tag: latest
  digest: sha256:8c851e7980d33de792af7cd87e1f6de8a9d592fb803c5d478a7980b70ae76dc3
runtime-dependency:
  registry: mcr.microsoft.com
  repository: dotnet/aspnet
  tag: "9.0"
  digest: sha256:1af4114db9ba87542a3f23dbb5cd9072cad7fcc8505f6e9131d1feb580286a6f
buildtime-dependency:
- registry: mcr.microsoft.com
  repository: dotnet/sdk
  tag: "9.0"
  digest: sha256:bb42ae2c058609d1746baf24fe6864ecab0686711dfca1f4b7a99e367ab17162
git: {}

```

Run ID: ca1 was successful after 51s

Note: Below is what the code means:

registry studentdemo3121acr → Your ACR name

image weather-app:latest → Tags the image inside ACR

Uses the Dockerfile in the current directory

Note: This avoids compatibility issues since the build runs directly in Azure.

3.6 Create an AKS Cluster with ACR Integration

Now let's create the Kubernetes cluster where our app will run:

```
az aks create \
  --resource-group student-demo \
  --name student-aks-cluster \
  --node-count 1 \
  --node-vm-size Standard_B2s \
  --attach-acr studentdemo2024acr \
  --enable-managed-identity \
  --generate-ssh-keys
```

❏ ❏ ❏ ❏

weather-app-demo > WeatherApp > Dockerfile > ...

1 # Multi-stage build for optimized image size

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS AZURE

```
● kosi@Mac WeatherApp % az aks create \
  --resource-group student-demo \
  --name student-aks-cluster \
  --node-count 1 \
  --node-vm-size Standard_B2s \
  --attach-acr studentdemo3121acr \
  --enable-managed-identity \
  --generate-ssh-keys
AAD role propagation done[#####] 100.0000%{
  "aadProfile": null,
  "addonProfiles": null,
  "agentPoolProfiles": [
    {
      "availabilityZones": null,
      "capacityReservationGroupId": null,
      "count": 1,
      "creationData": null,
      "currentOrchestratorVersion": "1.32.6",
      "eTag": null,
      "enableAutoScaling": false,
      "enableEncryptionAtHost": false,
      "enableFips": false,
      "enableNodePublicIp": false,
      "enableUltraSsd": false,
      "gatewayProfile": null,
      "gpuInstanceProfile": null,
      "gpuProfile": null,
      "hostGroupId": null,
      "kubeletConfig": null,
      "kubeletDiskType": "OS",
      "linuxOsConfig": null,
      "maxCount": null,
      "maxPods": 250,
      "messageOfTheDay": null,
      "minCount": null,
      "mode": "System",
      "name": "nodepool1",
      "networkProfile": null,
      "nodeImageVersion": "AKSUbuntu-2204gen2containerd-202508.20.1",
      "nodeLabels": null,
      "nodePublicIpPrefixId": null,
      "nodeTaints": null,
      "orchestratorVersion": "1.32",
      "osDiskSizeGb": 128,
      "osDiskType": "Managed",
      "osSku": "Ubuntu",
      "osType": "Linux",
      "podIpAllocationMode": null,
      "podSubnetId": null,
```

weather-app-demo > WeatherApp >  Dockerfile > ...

1 # Multi-stage build for optimized image size

PROBLEMS

OUTPUT

DEBUG CONSOLE

TERMINAL

PORTS

AZURE

```

    "powerState": {
      "code": "Running"
    },
    "provisioningState": "Succeeded",
    "proximityPlacementGroupId": null,
    "scaleDownMode": "Delete",
    "scaleSetEvictionPolicy": null,
    "scaleSetPriority": null,
    "securityProfile": {
      "enableSecureBoot": false,
      "enableVtpm": false
    },
    "spotMaxPrice": null,
    "status": null,
    "tags": null,
    "type": "VirtualMachineScaleSets",
    "upgradeSettings": {
      "drainTimeoutInMinutes": null,
      "maxSurge": "10%",
      "maxUnavailable": "0",
      "nodeSoakDurationInMinutes": null,
      "undrainableNodeBehavior": null
    },
    "virtualMachineNodesStatus": null,
    "virtualMachinesProfile": null,
    "vmSize": "Standard_B2s",
    "vnetSubnetId": null,
    "windowsProfile": null,
    "workloadRuntime": null
  }
},
"aiToolchainOperatorProfile": null,
"apiServerAccessProfile": null,
"autoScalerProfile": null,
"autoUpgradeProfile": {
  "nodeOsUpgradeChannel": "NodeImage",
  "upgradeChannel": null
},
"azureMonitorProfile": null,
"azurePortalFqdn": "student-ak-student-demo-1b7e34-yi6bn4ek.portal.hcp.eastus.azmk8s.io",
"bootstrapProfile": {
  "artifactSource": "Direct",
  "containerRegistryId": null
},
"currentKubernetesVersion": "1.32.6",
"disableLocalAccounts": false,
"diskEncryptionSetId": null,
"dnsPrefix": "student-ak-student-demo-1b7e34",
"eTag": null,

```

Dockerfile

weather-app-demo > WeatherApp > Dockerfile > ...

1 # Multi-stage build for optimized image size

PROBLEMS OUTPUT DEBUG CONSOLE **TERMINAL** PORTS AZURE

zsh - WeatherApp + - ...

```

"dnsprefix": "student-aks-student-demo-1b7e34",
"etag": null,
"enableRbac": true,
"extendedLocation": null,
"fqdn": "student-aks-student-demo-1b7e34-y16bn4ek.hcp.eastus.azmk8s.io",
"fqdnSubdomain": null,
"httpProxyConfig": null,
"id": "/subscriptions/1b7e34a5-85f8-4689-864f-5c27fe5306f5/resourcegroups/student-demo/providers/Microsoft.ContainerService/managedClusters/student-aks-cluster",
"identity": {
  "delegatedResources": null,
  "principalId": "29b371dd-3ba9-4829-b34d-bf6765b7fbda",
  "tenantId": "ec32aec3-a06f-441b-8910-226adb4d58de",
  "type": "SystemAssigned",
  "userAssignedIdentities": null
},
"identityProfile": {
  "kubeletIdentity": {
    "clientId": "66cd14ea-24a6-45b0-9e4f-1a4809cb35dc",
    "objectId": "8a9063ef-863e-4b3c-8d5c-4a9c938e6385",
    "resourceId": "/subscriptions/1b7e34a5-85f8-4689-864f-5c27fe5306f5/resourcegroups/MC_student-demo_student-aks-cluster_eastus/providers/Microsoft.ManagedIdentity/userAssignedIdentities/student-aks-cluster-agentpool"
  },
  "ingressProfile": null,
  "kind": "Base",
  "kubernetesVersion": "1.32",
  "linuxProfile": {
    "adminUsername": "azureuser",
    "ssh": {
      "publicKeys": [
        {
          "keyData": "ssh-rsa AAAAB3NzaC1yc2EAAAADAQABAAQ3Pabzp0g14E4pS/C0nGdXtSF+ZKfPWC1xyz9A7ZRfgfowPdfQtUNMCTd84o+AoGQHyK/CoanHGHBLgFyJ4aJ10rQb8u3Xk2stX4bzFgPZLlKHnSQR14N5x+qdwuc620ALVLfuhAE8ufLbgEuENpaYnz/Q5yvtiZ2zd1otZ2iiw6t1ZEjEfVKEUfSRkkSjhKgC639NuhZJYsAbV6+Xto06dyKbdkxZhDLDI1/xrsSSW8BzCTva07fcIFyd0QyaJ1m4uS0MnN4hrv00p55jtxJdS36Dg9AXghHz0dEzC05hzWjzu0hMVP0a8C2mVaxJkS00yFdrNe+3QYBYXkCsvr"
        }
      ]
    }
  },
  "location": "eastus",
  "maxAgentPools": 100,
  "metricsProfile": {
    "costAnalysis": {
      "enabled": false
    }
  },
  "name": "student-aks-cluster",
  "networkProfile": {
    "advancedNetworking": null,
    "dnsServiceIp": "10.0.0.10",

```

Dockerfile

weather-app-demo > WeatherApp > Dockerfile > ...

1 # Multi-stage build for optimized image size

PROBLEMS OUTPUT DEBUG CONSOLE **TERMINAL** PORTS AZURE

zsh - WeatherApp + - ...

```

"dnsserviceip": "10.0.0.10",
"ipfamilies": [
  "IPv4"
],
"loadBalancerProfile": {
  "allocatedOutboundPorts": null,
  "backendPoolType": "nodeIPConfiguration",
  "effectiveOutboundIPs": [
    {
      "id": "/subscriptions/1b7e34a5-85f8-4689-864f-5c27fe5306f5/resourcegroups/MC_student-demo_student-aks-cluster_eastus/providers/Microsoft.Network/publicIPAddresses/d26610fa-572c-469c-bee6-f836b4fc13ab",
      "resourceGroup": "MC_student-demo_student-aks-cluster_eastus"
    }
  ],
  "enableMultipleStandardLoadBalancers": null,
  "idleTimeoutInMinutes": null,
  "managedOutboundIPs": {
    "count": 1,
    "countIPv6": null
  },
  "outboundIPs": null,
  "outboundIPPrefixes": null
},
"loadBalancerSku": "standard",
"natGatewayProfile": null,
"networkDataplane": "azure",
"networkMode": null,
"networkPlugin": "azure",
"networkPluginMode": "overlay",
"networkPolicy": "none",
"outboundType": "loadBalancer",
"podCidr": "10.244.0.0/16",
"podCidrs": [
  "10.244.0.0/16"
],
"serviceCidr": "10.0.0.0/16",
"serviceCidrs": [
  "10.0.0.0/16"
],
"staticEgressGatewayProfile": null
},
"nodeProvisioningProfile": {
  "defaultNodePools": "Auto",
  "mode": "Manual"
},
"nodeResourceGroup": "MC_student-demo_student-aks-cluster_eastus",
"nodeResourceGroupProfile": null,
"oidcIssuerProfile": {
  "enabled": false,

```

```
weather-app-demo > WeatherApp > Dockerfile > ...
1 # Multi-stage build for optimized image size

{
  "code": "Running"
},
"privateFqdn": null,
"privateLinkResources": null,
"provisioningState": "Succeeded",
"publicNetworkAccess": null,
"resourceGroup": "student-demo",
"resourceId": "68c4341ee301eb0001f19be5",
"securityProfile": {
  "azureKeyVaultKms": null,
  "customCaTrustCertificates": null,
  "defender": null,
  "imageCleaner": null,
  "workloadIdentity": null
},
"serviceMeshProfile": null,
"servicePrincipalProfile": {
  "clientId": "msi",
  "secret": null
},
"sku": {
  "name": "Base",
  "tier": "Free"
},
"status": null,
"storageProfile": {
  "blobCsiDriver": null,
  "diskCsiDriver": {
    "enabled": true
  },
  "fileCsiDriver": {
    "enabled": true
  },
  "snapshotController": {
    "enabled": true
  }
},
"supportPlan": "KubernetesOfficial",
"systemData": null,
"tags": null,
"type": "Microsoft.ContainerService/ManagedClusters",
"upgradeSettings": null,
"windowsProfile": null,
"workloadAutoScalerProfile": {
  "keda": null,
  "verticalPodAutoscaler": null
}
}
```

What this does:

Creates a managed AKS cluster

Adds 1 VM node (2 vCPUs, 4 GB RAM)

Integrates ACR automatically (avoids ImagePullBackOff errors)

Generates SSH keys for secure access

Now wait 5–10 minutes for Azure to provision everything (coffee break time ☕).

3.7 Connect to Your AKS Cluster

Once the cluster is ready, configure kubectl to connect:

```
az aks get-credentials \
--resource-group student-demo \
--name student-aks-cluster
```

```
Dockerfile ×
weather-app-demo > WeatherApp > Dockerfile > ...
1 # Multi-stage build for optimized image size
```



```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS  AZURE

● kosi@Mac WeatherApp % az aks get-credentials \
  --resource-group student-demo \
  --name student-aks-cluster
Merged "student-aks-cluster" as current context in /Users/kosi/.kube/config
○ kosi@Mac WeatherApp %
```

Verify the connection:

kubectl get nodes

```
● kosi@Mac WeatherApp % kubectl get nodes
NAME                                STATUS    ROLES    AGE    VERSION
aks-nodepool1-12794462-vmss000000  Ready    <none>   8m19s  v1.32.6
○ kosi@Mac WeatherApp %
```

You should see your Kubernetes node(s) listed.

3.8 Verify ACR Integration (Optional)

Check that your AKS cluster can pull images from ACR:

```
CLIENT_ID=$(az aks show \
  --resource-group student-demo \
  --name student-aks-cluster \
  --query "identityProfile.kubeletidentity.clientId" \
  --output tsv)

echo "Kubelet Client ID: $CLIENT_ID"
```

❏ ❏ ❏ ❏

```
Dockerfile ×

weather-app-demo > WeatherApp > Dockerfile > ...
1  # Multi-stage build for optimized image size
2

PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS  AZURE

● kosi@Mac WeatherApp % CLIENT_ID=$(az aks show \
  --resource-group student-demo \
  --name student-aks-cluster \
  --query "identityProfile.kubeletidentity.clientId" \
  --output tsv)
```

```
--output tsv)
● kosi@Mac WeatherApp % echo "Kubelet Client ID: $CLIENT_ID"
  Kubelet Client ID: 66cd14ea-24a6-45b0-9e4f-1a4809cb35dc
○ kosi@Mac WeatherApp %
```

Now confirm the role assignment:

```
az role assignment list \
--assignee $CLIENT_ID \
--scope $(az acr show --name studentdemo3121acr --query id --output tsv) \
--output table
```

```
● kosi@Mac WeatherApp % az role assignment list \
  --assignee $CLIENT_ID \
  --scope $(az acr show --name studentdemo3121acr --query id --output tsv) \
  --output table
Principal                                     Role    Scope
-----
-----
```

You should see an AcrPull role. If not, create it manually:

```
az role assignment create \
--assignee $CLIENT_ID \
--role AcrPull \
--scope $(az acr show --name studentdemo3121acr --query id --output tsv)
```

At this point, you have a fully provisioned Azure Kubernetes cluster and a container image stored in ACR. Next, we will deploy the app into Kubernetes.

Step 4: Create Kubernetes Configuration

Now that we have our AKS cluster and image in ACR, it's time to define the Kubernetes manifests that describe how our app should run in the cluster.

4.1 Set Up a Manifests Directory

Organize your Kubernetes YAML files inside a k8s folder:

```
cd .. to Go back to weather-app-demo root
mkdir k8s
cd k8s
```

```
● kosi@Mac WeatherApp % cd ..
● kosi@Mac weather-app-demo % mkdir k8s
● kosi@Mac weather-app-demo % cd k8s
○ kosi@Mac k8s %
```

4.2 Create the Deployment

A Deployment ensures your app runs in Kubernetes with the desired number of replicas, health checks, and resource limits.

Create a file called deployment.yaml:

`touch .deployment.yaml` - This creates a file

Now copy and paste the code below into the `deployment.yaml` file created

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: weather-app
  labels:
    app: weather-app
    version: v1
spec:
  replicas: 2
  selector:
    matchLabels:
      app: weather-app
  template:
    metadata:
      labels:
        app: weather-app
        version: v1
    spec:
      containers:
        - name: weather-app
          image: studentdemo2024acr.azurecr.io/weather-app:latest
          imagePullPolicy: Always
          ports:
```

```
- name: http
  containerPort: 8080
  protocol: TCP
env:
- name: ASPNETCORE_ENVIRONMENT
  value: "Production"
resources:
  requests:
    memory: "128Mi"
    cpu: "100m"
  limits:
    memory: "512Mi"
    cpu: "500m"
livenessProbe:
  httpGet:
    path: /health
    port: http
  initialDelaySeconds: 30
  periodSeconds: 10
  timeoutSeconds: 5
  failureThreshold: 3
readinessProbe:
  httpGet:
    path: /health
    port: http
  initialDelaySeconds: 10
  periodSeconds: 5
  timeoutSeconds: 3
  failureThreshold: 3
startupProbe:
  httpGet:
    path: /health
    port: http
  initialDelaySeconds: 10
  periodSeconds: 5
  timeoutSeconds: 3
  failureThreshold: 30
```



The screenshot shows the Visual Studio Code interface. The Explorer sidebar on the left displays the project structure for 'Week 2 DevOps Project', including folders like 'weather-app-demo' and 'k8s', and files like 'deployment.yaml'. The main editor area shows the content of 'deployment.yaml', which is a Kubernetes Deployment manifest for a 'weather-app'. The manifest includes metadata (name, labels), a spec with 2 replicas, and a template for the container image 'studentdemo2024acr.azurecr.io/weather-app:latest' on port 8080. The bottom panel shows the 'TERMINAL' tab with a series of shell commands executed in a 'k8s' directory: creating the directory, navigating to it, creating the file, and touching it.

```
weather-app-demo > k8s > ! deployment.yaml > {} spec > {} template > {} spec > [ ] containers > {} 0 > {} start
io.k8s.api.apps.v1.Deployment (v1@deployment.json)
1  apiVersion: apps/v1
2  kind: Deployment
3  metadata:
4    name: weather-app
5    labels:
6      app: weather-app
7      version: v1
8  spec:
9    replicas: 2
10   selector:
11     matchLabels:
12       app: weather-app
13   template:
14     metadata:
15       labels:
16         app: weather-app
17         version: v1
18   spec:
19     containers:
20     - name: weather-app
21       image: studentdemo2024acr.azurecr.io/weather-app:latest
22       imagePullPolicy: Always
23       ports:
24       - name: http
25         containerPort: 8080
26         protocol: TCP
27       env:
28       - name: ASPNETCORE_ENVIRONMENT
```

```
kosi@Mac WeatherApp % cd ..
kosi@Mac weather-app-demo % mkdir k8s
kosi@Mac weather-app-demo % cd k8s
kosi@Mac k8s % touch deployment.yaml
kosi@Mac k8s %
```

Notes:

Uses 2 replicas for high availability.

Includes liveness, readiness, and startup probes to let Kubernetes know when the app is healthy.

No imagePullSecrets needed because AKS was created with --attach-acr to handle authentication automatically!

4.3 Create the Service

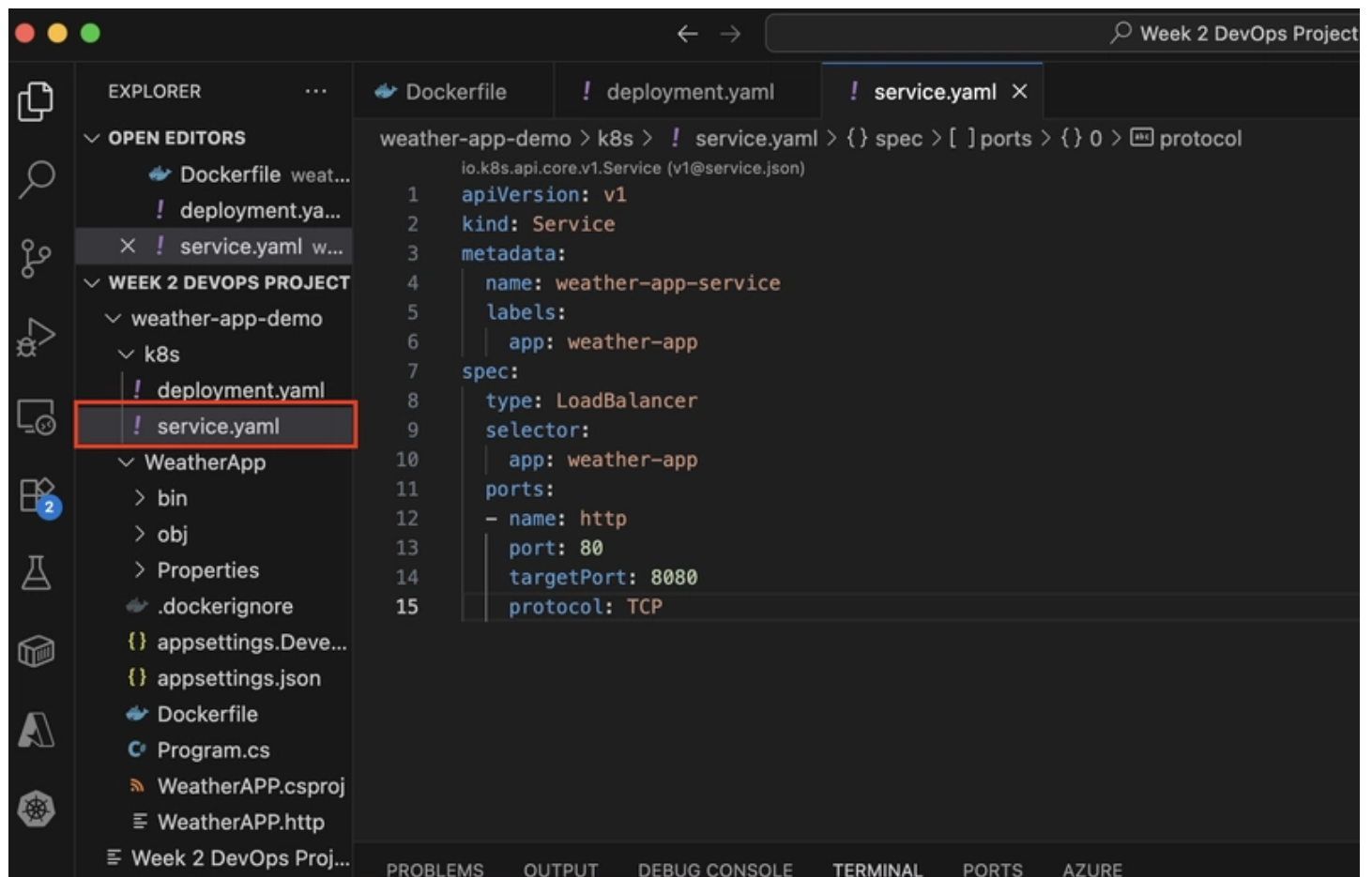
A Service exposes your app to the internet through a cloud load balancer.

Create a file called service.yaml:

Same process we used in 4.2

```
apiVersion: v1
kind: Service
metadata:
  name: weather-app-service
  labels:
    app: weather-app
spec:
  type: LoadBalancer
  selector:
    app: weather-app
  ports:
    - name: http
      port: 80
      targetPort: 8080
      protocol: TCP
```

❏ ❏ ❏ ❏




```
● kosi@Mac k8s % touch service.yaml
○ kosi@Mac k8s %
```

4.4 Deploy to Kubernetes

Apply both manifests to your cluster:

```
kubectl apply -f deployment.yaml
```

```
kubectl apply -f service.yaml
```

Check status:

```
kubectl get deployments
```

```
kubectl get pods
```

```
kubectl get services
```

Watch pods come online:

```
kubectl get pods --watch
```

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS AZURE
● kosi@Mac weather-app-demo % kubectl get deployments
NAME READY UP-TO-DATE AVAILABLE AGE
weather-app 2/2 2 2 36m
● kosi@Mac weather-app-demo % kubectl get pods
NAME READY STATUS RESTARTS AGE
weather-app-5db4dbfd45-2bnrl 1/1 Running 0 18m
weather-app-5db4dbfd45-mgpcs 1/1 Running 0 18m
● kosi@Mac weather-app-demo % kubectl get services
NAME TYPE CLUSTER-IP EXTERNAL-IP PORT(S) AGE
kubernetes ClusterIP 10.0.0.1 <none> 443/TCP 23h
weather-app-service LoadBalancer 10.0.247.196 48.216.149.220 80:32413/TCP 36m
○ kosi@Mac weather-app-demo % kubectl get pods --watch
NAME READY STATUS RESTARTS AGE
weather-app-5db4dbfd45-2bnrl 1/1 Running 0 19m
weather-app-5db4dbfd45-mgpcs 1/1 Running 0 19m
□
```

Once ready, your service will show an external IP, use that to access the app in your browser.

```
● kosi@Mac weather-app-demo % kubectl get service weather-app-service
NAME TYPE CLUSTER-IP EXTERNAL-IP PORT(S) AGE
weather-app-service LoadBalancer 10.0.247.196 48.216.149.220 80:32413/TCP 37m
○ kosi@Mac weather-app-demo %
```



Not Secure

48.216.149.220

Pretty print ☒

```
{
  "message": "Welcome to the Weather App!",
  "version": "1.0.0",
  "environment": "Production",
  "timestamp": "2025-09-13T14:02:04.947122Z"
}
```

4.5 Troubleshooting Tips

If you see **ImagePullBackOff**, debug like this:

Describe pod to see error details

```
kubectl describe pod $(kubectl get pods -l app=weather-app -o
jsonpath='{.items[0].metadata.name}')
```

Verify the image exists in ACR

```
az acr repository show-tags --name studentdemo2024acr --repository weather-app --
output table
```

Restart deployment (forces a new image pull)

```
kubectl rollout restart deployment/weather-app
```

At this point, your app should be running on AKS and accessible from the public LoadBalancer IP.

Step 5: Set Up GitHub Actions CI/CD

With our app running in AKS, the final step is to automate deployment using GitHub Actions.

This way, every time you push changes to your repo, GitHub will:

Build the .NET app.

Build and push a Docker image to Azure Container Registry (ACR).

Update the Kubernetes deployment in AKS.

5.1 Initialize Git Repository

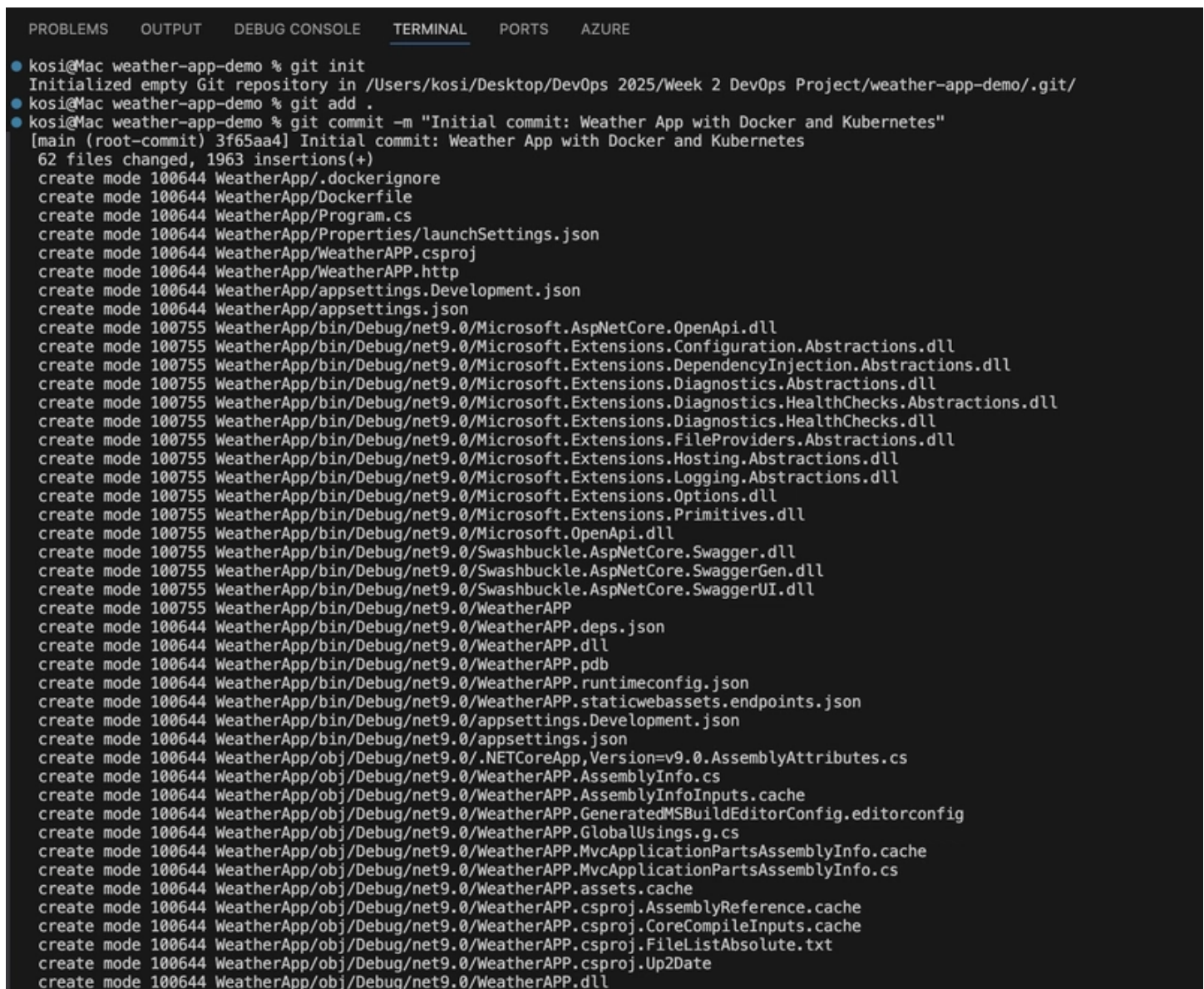
If you haven't already:

cd .. ### Back to weather-app-demo folder

```
git init
```

```
git add .
```

```
git commit -m "Initial commit: Weather App with Docker and Kubernetes"
```



The screenshot shows a terminal window with the following output:

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS  AZURE

● kosi@Mac weather-app-demo % git init
Initialized empty Git repository in /Users/kosi/Desktop/DevOps 2025/Week 2 DevOps Project/weather-app-demo/.git/
● kosi@Mac weather-app-demo % git add .
● kosi@Mac weather-app-demo % git commit -m "Initial commit: Weather App with Docker and Kubernetes"
[main (root-commit) 3f65aa4] Initial commit: Weather App with Docker and Kubernetes
62 files changed, 1963 insertions(+)
create mode 100644 WeatherApp/.dockerignore
create mode 100644 WeatherApp/Dockerfile
create mode 100644 WeatherApp/Program.cs
create mode 100644 WeatherApp/Properties/launchSettings.json
create mode 100644 WeatherApp/WeatherAPP.csproj
create mode 100644 WeatherApp/WeatherAPP.http
create mode 100644 WeatherApp/appsettings.Development.json
create mode 100644 WeatherApp/appsettings.json
create mode 100755 WeatherApp/bin/Debug/net9.0/Microsoft.AspNetCore.OpenApi.dll
create mode 100755 WeatherApp/bin/Debug/net9.0/Microsoft.Extensions.Configuration.Abstractions.dll
create mode 100755 WeatherApp/bin/Debug/net9.0/Microsoft.Extensions.DependencyInjection.Abstractions.dll
create mode 100755 WeatherApp/bin/Debug/net9.0/Microsoft.Extensions.Diagnostics.Abstractions.dll
create mode 100755 WeatherApp/bin/Debug/net9.0/Microsoft.Extensions.Diagnostics.HealthChecks.Abstractions.dll
create mode 100755 WeatherApp/bin/Debug/net9.0/Microsoft.Extensions.Diagnostics.HealthChecks.dll
create mode 100755 WeatherApp/bin/Debug/net9.0/Microsoft.Extensions.FileProviders.Abstractions.dll
create mode 100755 WeatherApp/bin/Debug/net9.0/Microsoft.Extensions.Hosting.Abstractions.dll
create mode 100755 WeatherApp/bin/Debug/net9.0/Microsoft.Extensions.Logging.Abstractions.dll
create mode 100755 WeatherApp/bin/Debug/net9.0/Microsoft.Extensions.Options.dll
create mode 100755 WeatherApp/bin/Debug/net9.0/Microsoft.Extensions.Primitives.dll
create mode 100755 WeatherApp/bin/Debug/net9.0/Microsoft.OpenApi.dll
create mode 100755 WeatherApp/bin/Debug/net9.0/Swashbuckle.AspNetCore.Swagger.dll
create mode 100755 WeatherApp/bin/Debug/net9.0/Swashbuckle.AspNetCore.SwaggerGen.dll
create mode 100755 WeatherApp/bin/Debug/net9.0/Swashbuckle.AspNetCore.SwaggerUI.dll
create mode 100755 WeatherApp/bin/Debug/net9.0/WeatherAPP
create mode 100644 WeatherApp/bin/Debug/net9.0/WeatherAPP.deps.json
create mode 100644 WeatherApp/bin/Debug/net9.0/WeatherAPP.dll
create mode 100644 WeatherApp/bin/Debug/net9.0/WeatherAPP.pdb
create mode 100644 WeatherApp/bin/Debug/net9.0/WeatherAPP.runtimeconfig.json
create mode 100644 WeatherApp/bin/Debug/net9.0/WeatherAPP.staticwebassets.endpoints.json
create mode 100644 WeatherApp/bin/Debug/net9.0/appsettings.Development.json
create mode 100644 WeatherApp/bin/Debug/net9.0/appsettings.json
create mode 100644 WeatherApp/obj/Debug/net9.0/.NETCoreApp,Version=v9.0.AssemblyAttributes.cs
create mode 100644 WeatherApp/obj/Debug/net9.0/WeatherAPP.AssemblyInfo.cs
create mode 100644 WeatherApp/obj/Debug/net9.0/WeatherAPP.AssemblyInfoInputs.cache
create mode 100644 WeatherApp/obj/Debug/net9.0/WeatherAPP.GeneratedMSBuildEditorConfig.editorconfig
create mode 100644 WeatherApp/obj/Debug/net9.0/WeatherAPP.GlobalUsings.g.cs
create mode 100644 WeatherApp/obj/Debug/net9.0/WeatherAPP.MvcApplicationPartsAssemblyInfo.cache
create mode 100644 WeatherApp/obj/Debug/net9.0/WeatherAPP.MvcApplicationPartsAssemblyInfo.cs
create mode 100644 WeatherApp/obj/Debug/net9.0/WeatherAPP.assets.cache
create mode 100644 WeatherApp/obj/Debug/net9.0/WeatherAPP.csproj.AssemblyReference.cache
create mode 100644 WeatherApp/obj/Debug/net9.0/WeatherAPP.csproj.CoreCompileInputs.cache
create mode 100644 WeatherApp/obj/Debug/net9.0/WeatherAPP.csproj.FileListAbsolute.txt
create mode 100644 WeatherApp/obj/Debug/net9.0/WeatherAPP.csproj.Up2Date
create mode 100644 WeatherApp/obj/Debug/net9.0/WeatherAPP.dll
```

5.2 Create Azure Service Principal

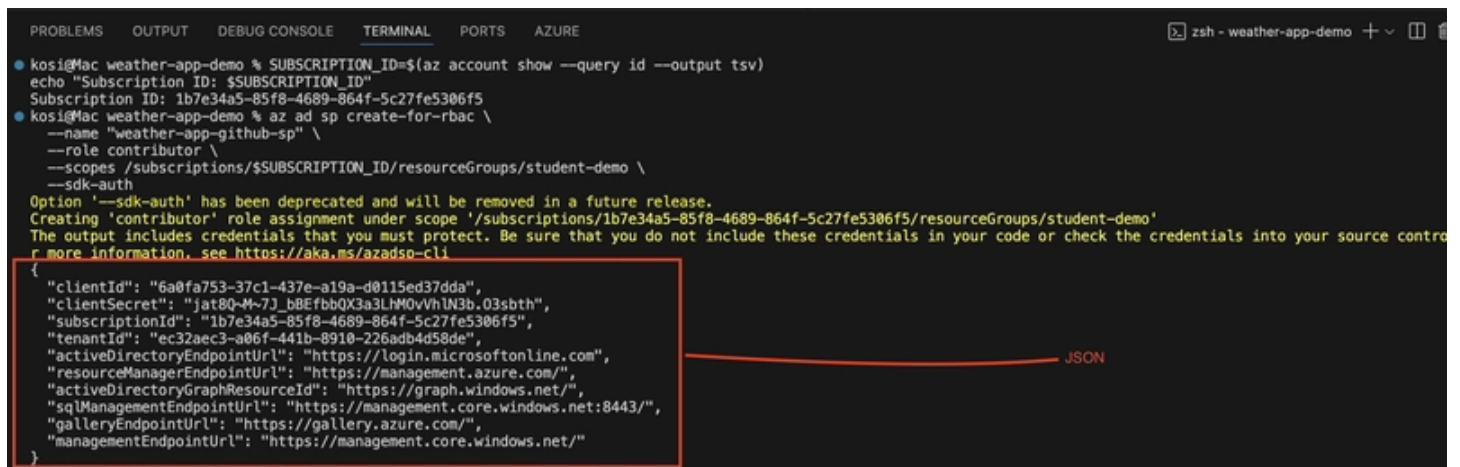
GitHub Actions needs credentials to interact with Azure. For that, we will create a Service Principal (SP).

Get your subscription ID:

```
SUBSCRIPTION_ID=$(az account show --query id --output tsv)
echo "Subscription ID: $SUBSCRIPTION_ID"
```

Create the service principal:

```
az ad sp create-for-rbac \
--name "weather-app-github-sp" \
--role contributor \
--scopes /subscriptions/$SUBSCRIPTION_ID/resourceGroups/student-demo \
--sdk-auth
```



```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS AZURE
kosi@Mac weather-app-demo % SUBSCRIPTION_ID=$(az account show --query id --output tsv)
echo "Subscription ID: $SUBSCRIPTION_ID"
Subscription ID: 1b7e34a5-85f8-4689-864f-5c27fe5306f5
kosi@Mac weather-app-demo % az ad sp create-for-rbac \
--name "weather-app-github-sp" \
--role contributor \
--scopes /subscriptions/$SUBSCRIPTION_ID/resourceGroups/student-demo \
--sdk-auth
Option '--sdk-auth' has been deprecated and will be removed in a future release.
Creating 'contributor' role assignment under scope '/subscriptions/1b7e34a5-85f8-4689-864f-5c27fe5306f5/resourceGroups/student-demo'
The output includes credentials that you must protect. Be sure that you do not include these credentials into your source control
for more information, see https://aka.ms/azaso-cli
{
  "clientId": "6a0fa753-37c1-437e-a19a-d0115ed37dda",
  "clientSecret": "jat8Q-M-7J_bBEfbQX3a3LHM0vVhIN3b.03sbth",
  "subscriptionId": "1b7e34a5-85f8-4689-864f-5c27fe5306f5",
  "tenantId": "ec32aec3-a06f-441b-8910-226adb4d58de",
  "activeDirectoryEndpointUrl": "https://login.microsoftonline.com/",
  "resourceManagerEndpointUrl": "https://management.azure.com/",
  "activeDirectoryGraphResourceId": "https://graph.windows.net/",
  "sqlManagementEndpointUrl": "https://management.core.windows.net:8443/",
  "galleryEndpointUrl": "https://gallery.azure.com/",
  "managementEndpointUrl": "https://management.core.windows.net/"
}
```

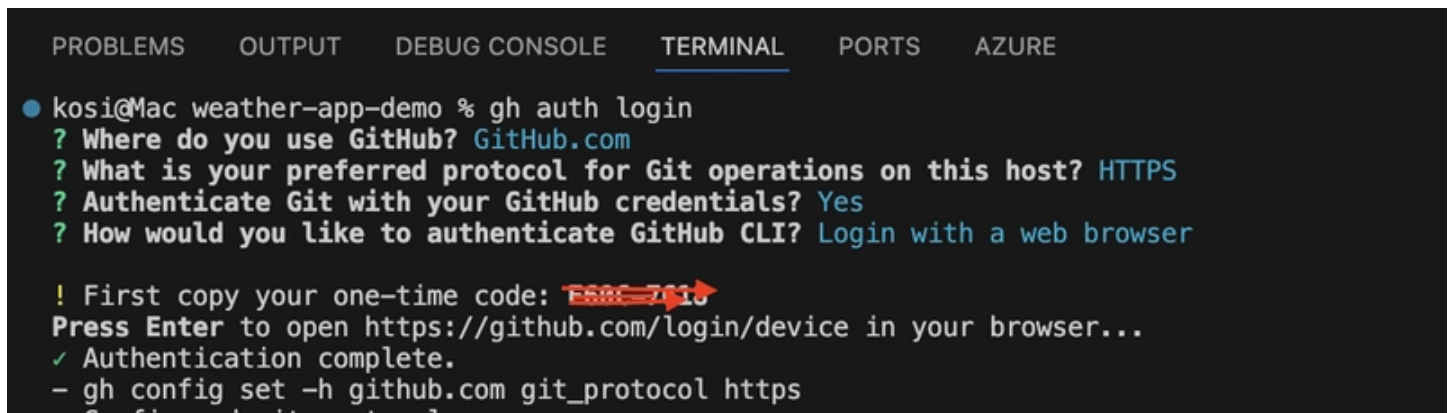
Important: Copy the full JSON output. You will paste it into GitHub Secrets in a moment.

5.3 Create GitHub Repository

You can create a repo either via GitHub CLI:

```
gh auth login ### Follow the prompts
```

```
gh repo create weather-app-demo --public --source=. --push
```



```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS AZURE
kosi@Mac weather-app-demo % gh auth login
? Where do you use GitHub? GitHub.com
? What is your preferred protocol for Git operations on this host? HTTPS
? Authenticate Git with your GitHub credentials? Yes
? How would you like to authenticate GitHub CLI? Login with a web browser

! First copy your one-time code: F58C 7818
Press Enter to open https://github.com/login/device in your browser...
✓ Authentication complete.
- gh config set -h github.com git_protocol https
- Configured git protocol
```



```
✓ Configured git protocol
✓ Logged in as kosinachi
```

A new release of gh is available: 2.78.0 → 2.79.0

To upgrade, run: brew upgrade gh

<https://github.com/cli/cli/releases/tag/v2.79.0>

```
• kosi@Mac weather-app-demo % gh repo create weather-app-demo --public --source=. --push
✓ Created repository kosinachi/weather-app-demo on github.com
  https://github.com/kosinachi/weather-app-demo
✓ Added remote https://github.com/kosinachi/weather-app-demo.git
Enumerating objects: 66, done.
Counting objects: 100% (66/66), done.
Delta compression using up to 10 threads
Compressing objects: 100% (59/59), done.
Writing objects: 100% (66/66), 1.12 MiB | 3.93 MiB/s, done.
Total 66 (delta 10), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (10/10), done.
To https://github.com/kosinachi/weather-app-demo.git
 * [new branch]      HEAD -> main
branch 'main' set up to track 'origin/main'.
✓ Pushed commits to https://github.com/kosinachi/weather-app-demo.git
○ kosi@Mac weather-app-demo %
```

The screenshot shows the GitHub repository page for 'weather-app-demo' by user 'kosinachi'. The repository is public. At the top, there are buttons for 'Pin', 'Watch' (0), and 'Code'. Below the repository name, it shows 'main' branch, '1 Branch', and '0 Tags'. There is a search bar 'Go to file' and buttons for 'Add file' and 'Code'. The commit history shows one commit by 'kosinachi' titled 'Initial commit: Weather App with Docker and Kubernetes' at '36 minutes ago'. Below the commit, there are two folders listed: 'WeatherApp' and 'k8s', both with the same commit message and time.

Commit	Message	Time
3f65aa4	Initial commit: Weather App with Docker and Kubernetes	36 minutes ago

File	Message	Time
WeatherApp	Initial commit: Weather App with Docker and Kubernetes	36 minutes ago
k8s	Initial commit: Weather App with Docker and Kubernetes	36 minutes ago

Or manually on GitHub.com → then push your local repo.

5.4 Configure GitHub Secrets

In your repo, go to:

Settings → Secrets and variables → Actions → New repository secret

Add the following secrets:

Secret Name Value

AZURE_CREDENTIALS The full JSON output from Step 5.2

ACR_NAME studentdemo3121acr (your ACR name)

RESOURCE_GROUP student-demo

CLUSTER_NAME student-aks-cluster

OR configure in gh CLI

Set AZURE_CREDENTIALS (the JSON from step 5.2): `gh secret set AZURE_CREDENTIALS`

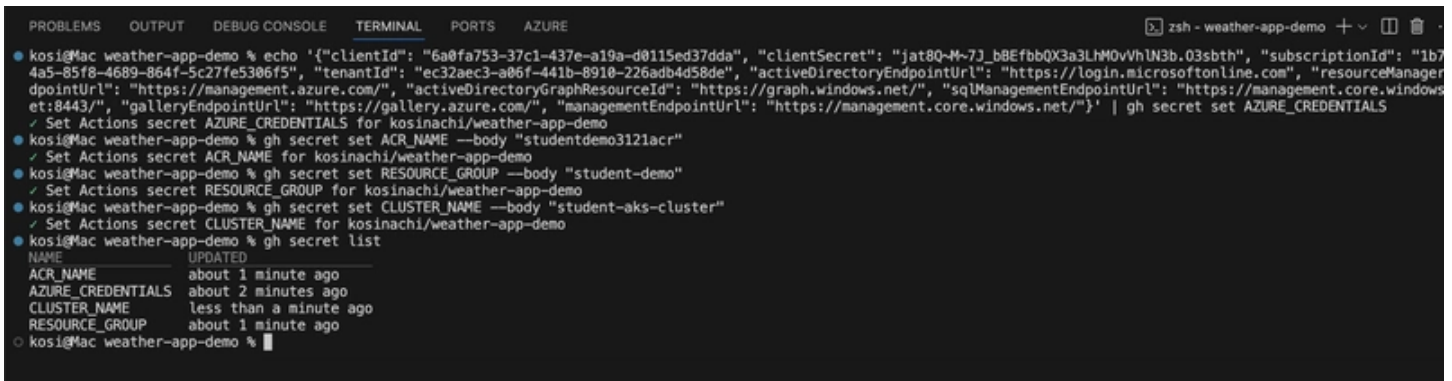
This will open an interactive prompt where you can paste your JSON credentials from step 5.2

Set the other secrets:

```
gh secret set ACR_NAME --body "studentdemo3121acr"
```

```
gh secret set RESOURCE_GROUP --body "student-demo"
```

```
gh secret set CLUSTER_NAME --body "student-aks-cluster"
```



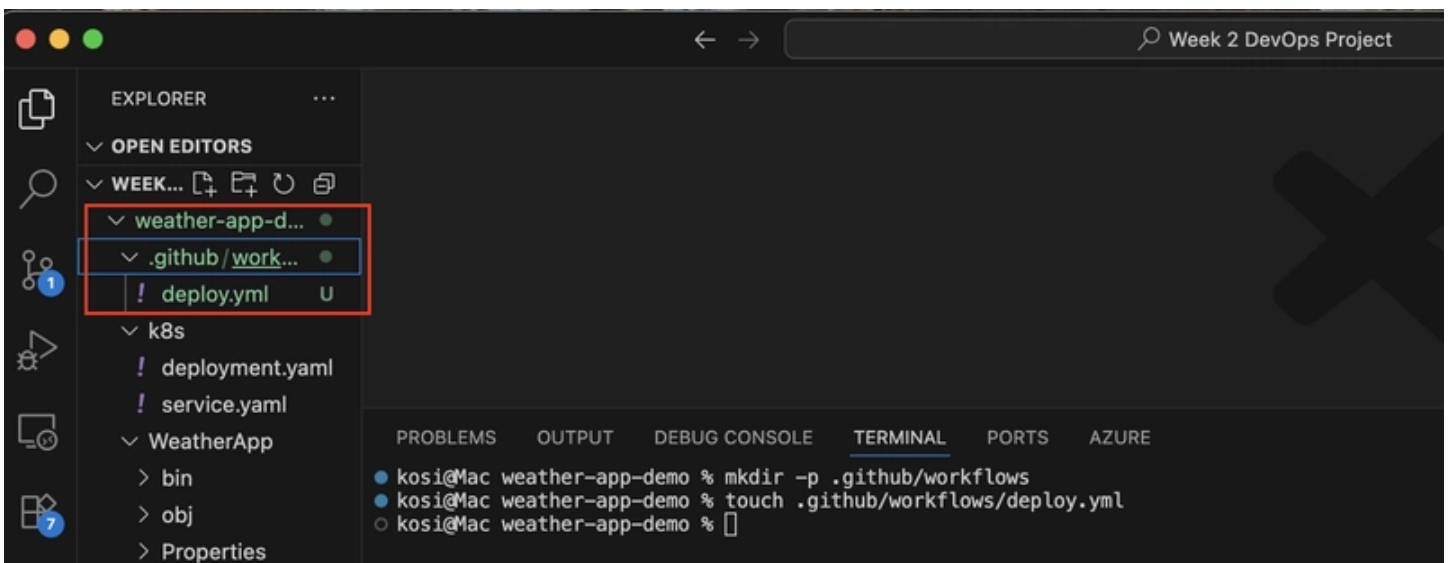
```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS AZURE
kosi@Mac weather-app-demo % echo '{"clientId": "6a0fa753-37c1-437e-a19a-d0115ed37dda", "clientSecret": "jat8Q-M-7J_bBEfbQX3a3LHM0vVhIN3b.03sbth", "subscriptionId": "1b74a5-85f8-4689-864f-5c27fe5306f5", "tenantId": "ec32aec3-a06f-441b-8910-226adb4d58de", "activeDirectoryEndpointUrl": "https://login.microsoftonline.com", "resourceManagerEndpointUrl": "https://management.azure.com/", "activeDirectoryGraphResourceId": "https://graph.windows.net/", "sqlManagementEndpointUrl": "https://management.core.windows.net:8443/", "galleryEndpointUrl": "https://gallery.azure.com/", "managementEndpointUrl": "https://management.core.windows.net/"}' | gh secret set AZURE_CREDENTIALS
✓ Set Actions secret AZURE_CREDENTIALS for kosi@Mac weather-app-demo
kosi@Mac weather-app-demo % gh secret set ACR_NAME --body "studentdemo3121acr"
✓ Set Actions secret ACR_NAME for kosi@Mac weather-app-demo
kosi@Mac weather-app-demo % gh secret set RESOURCE_GROUP --body "student-demo"
✓ Set Actions secret RESOURCE_GROUP for kosi@Mac weather-app-demo
kosi@Mac weather-app-demo % gh secret set CLUSTER_NAME --body "student-aks-cluster"
✓ Set Actions secret CLUSTER_NAME for kosi@Mac weather-app-demo
kosi@Mac weather-app-demo % gh secret list
NAME                UPDATED
ACR_NAME             about 1 minute ago
AZURE_CREDENTIALS    about 2 minutes ago
CLUSTER_NAME         less than a minute ago
RESOURCE_GROUP       about 1 minute ago
kosi@Mac weather-app-demo %
```

5.5 Create GitHub Workflow

Now add the GitHub Actions workflow.

```
mkdir -p .github/workflows
```

```
touch .github/workflows/deploy.yml
```



Paste the code below into `.github/workflows/deploy.yml`:


```
name: Build and Deploy to AKS
on:
  push:
    branches: [ main ]
  pull_request:
    branches: [ main ]

env:
  IMAGE_NAME: weather-app

jobs:
  build-and-deploy:
    runs-on: ubuntu-latest
    steps:
      - name: Checkout code
        uses: actions/checkout@v4

      - name: Setup .NET
        uses: actions/setup-dotnet@v3
        with:
          dotnet-version: '9.0.x'

      - name: Restore dependencies
        run: dotnet restore WeatherApp/WeatherAPP.csproj

      - name: Build application
        run: dotnet build WeatherApp/WeatherAPP.csproj --configuration Release --no-restore

      - name: Run tests
        run: dotnet test WeatherApp/WeatherAPP.csproj --no-build --verbosity normal || true

      - name: Azure Login
        uses: azure/login@v1
        with:
          creds: ${ secrets.AZURE_CREDENTIALS }
```

```

- name: Setup Azure CLI
  uses: azure/cli@v2
  with:
    inlineScript: echo "Azure CLI setup complete"

- name: Build and push Docker image to ACR
  run: |
    IMAGE_TAG=${{ secrets.ACR_NAME }}.azurecr.io/${{ env.IMAGE_NAME }}:${{
github.sha }}
    az acr build \
      --registry ${{{ secrets.ACR_NAME }}} \
      --image ${{{ env.IMAGE_NAME }}}:${{{ github.sha }}} \
      --file WeatherApp/Dockerfile \
      .
    echo "IMAGE_TAG=$IMAGE_TAG" >> $GITHUB_ENV

- name: Deploy to AKS
  if: github.ref == 'refs/heads/main'
  run: |
    echo "Checking if AKS cluster exists..."
    if ! az aks show --resource-group "${{{ secrets.RESOURCE_GROUP }}" --
name "${{{ secrets.CLUSTER_NAME }}" --output table; then
      echo "ERROR: AKS cluster '${{{ secrets.CLUSTER_NAME }}}' not found in
resource group '${{{ secrets.RESOURCE_GROUP }}}'"
      exit 1
    fi

    echo "Getting AKS credentials..."
    az aks get-credentials \
      --resource-group ${{{ secrets.RESOURCE_GROUP }}} \
      --name ${{{ secrets.CLUSTER_NAME }}} \
      --overwrite-existing

    echo "Testing kubectl connection..."
    kubectl cluster-info

```

```

echo "Updating deployment with image: ${ env.IMAGE_TAG }"
kubectl set image deployment/weather-app \
    weather-app=${ env.IMAGE_TAG }

echo "Waiting for rollout to complete..."
kubectl rollout status deployment/weather-app --timeout=600s

echo "Deployment status:"
kubectl get pods -l app=weather-app
kubectl get service weather-app-service

```

❏ ❏ ❏ ❏

```

! deploy.yml U X
weather-app-demo > .github > workflows > ! deploy.yml > {} jobs > {} build-and-deploy > [ ] steps > {} 8 > run
GitHub Workflow - YAML GitHub Workflow (github-workflow.json)
1  name: Build and Deploy to AKS
2
3  on:
4    push:
5      branches: [ main ]
6    pull_request:
7      branches: [ main ]
8
9  env:
10   IMAGE_NAME: weather-app
11
12  jobs:
13    build-and-deploy:
14      runs-on: ubuntu-latest
15      steps:
16        - name: Checkout code
17          uses: actions/checkout@v4

```

5.6 Deploy Your Application

Commit and push:

```

git add .
git commit -m "Add GitHub Actions CI/CD pipeline"
git push origin main

```

```

PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS  AZURE

● kosi@Mac weather-app-demo % git add .
● kosi@Mac weather-app-demo % git commit -m "Add GitHub Actions CI/CD pipeline"
[main bcc44ff] Add GitHub Actions CI/CD pipeline
1 file changed, 80 insertions(+)
create mode 100644 .github/workflows/deploy.yml
● kosi@Mac weather-app-demo % git push origin main
Enumerating objects: 6, done.
Counting objects: 100% (6/6), done.

```

```
Delta compression using up to 10 threads
Compressing objects: 100% (3/3), done.
Writing objects: 100% (5/5), 1.34 KiB | 1.34 MiB/s, done.
Total 5 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To https://github.com/kosinachi/weather-app-demo.git
   3f65aa4..bcc44ff  main -> main
o kosi@Mac weather-app-demo %
```

Now open your GitHub repo → Actions tab.

You will see the workflow running, building your Docker image, pushing to ACR, and updating your AKS cluster.

The screenshot displays the GitHub Actions interface for the repository 'weather-app-demo'. The main heading is 'Build and Deploy to AKS'. Below it, a workflow run is shown with the title 'Second Fix of Dockerfile case mismatch and clean build artifacts #4'. The status is 'Success' and the total duration is '2m 36s'. The workflow file 'deploy.yml' is shown, triggered on push. The left sidebar contains links for Summary, Jobs, Run details, Usage, and Workflow file. The top navigation bar includes links for Code, Issues, Pull requests, Actions, Projects, Wiki, Security, Insights, and Settings.

And just like that, we have built a full CI/CD pipeline for your .NET 8 app on Azure Kubernetes Service.

Step 6: Access Your Deployed Application

Your app is now deployed on Azure Kubernetes Service. Let's get the public URL so you can test it live.

6.1 Get the External IP Address

Check the status of your Kubernetes service:

```
kubectl get service weather-app-service
```

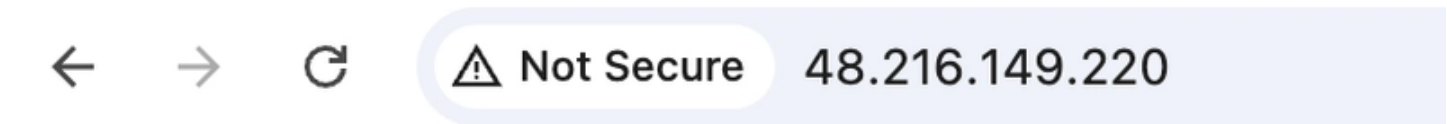
You will see output like this:

```
NAME TYPE CLUSTER-IP EXTERNAL-IP PORT(S) AGE
weather-app-service LoadBalancer 10.0.123.45 20.85.102.11 80:31000/TCP 2m
```

👉 The EXTERNAL-IP column is what you will use.
It may take 2–5 minutes to show up. Watch until it's assigned:

```
kubectl get service weather-app-service --watch
```

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS AZURE
• kosi@Mac weather-app-demo % kubectl get service weather-app-service
NAME TYPE CLUSTER-IP EXTERNAL-IP PORT(S) AGE
weather-app-service LoadBalancer 10.0.247.196 48.216.149.220 80:32413/TCP 23h
○ kosi@Mac weather-app-demo % kubectl get service weather-app-service --watch
NAME TYPE CLUSTER-IP EXTERNAL-IP PORT(S) AGE
weather-app-service LoadBalancer 10.0.247.196 48.216.149.220 80:32413/TCP 23h
█
```



pretty print ☒

```
{
  "message": "Welcome to the Weather App!",
  "version": "1.0.0",
  "environment": "Production",
  "timestamp": "2025-09-14T12:38:44.6038595Z"
}
```

6.2 Test Your Live Application

Once you have the external IP, test the endpoints:

For the Welcome message:

```
curl http://YOUR-EXTERNAL-IP/
```

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS AZURE
• kosi@Mac weather-app-demo % curl http://48.216.149.220/
```

```
kosi@Mac weather-app-demo % curl http://10.210.115.220/
{"message":"Welcome to the Weather App!","version":"1.0.0","environment":"Production","timestamp":"2025-09-14T12:40:15.5412044Z"}
kosi@Mac weather-app-demo %
```

Weather forecast:

```
curl http://YOUR-EXTERNAL-IP/weather
```

```

❯ kosi@mac weather-app-demo % curl http://4216.149.220.weather
{"date": "2025-09-15", "temperatureC": 41, "temperatureF": 105, "summary": "Mild"}, {"date": "2025-09-16", "temperatureC": 45, "temperatureF": 112, "summary": "Sweltering"}, {"date": "2025-09-17", "temperatureC": 30, "temperatureF": 85, "summary": "Mild"}, {"date": "2025-09-18", "temperatureC": -17, "temperatureF": 2, "summary": "Bracing"}, {"date": "2025-09-19", "temperatureC": 50, "temperatureF": 121, "summary": "Sweltering"}]
❯ kosi@mac weather-app-demo %

```

Health check:

```
curl http://YOUR-EXTERNAL-IP/health
```

```
● kosi@Mac weather-app-demo % curl http://48.216.149.220/health
Healthy%
○ kosi@Mac weather-app-demo %
```

Or just open it in your browser:

Pretty print ☒

```
[
  {
    "date": "2025-09-15",
    "temperatureC": 40,
    "temperatureF": 103,
    "summary": "Bracing"
  },
  {
    "date": "2025-09-16",
    "temperatureC": 1,
    "temperatureF": 33,
    "summary": "Bracing"
  },
  {
```



```
{
  "date": "2025-09-17",
  "temperatureC": 52,
  "temperatureF": 125,
  "summary": "Scorching"
},
{
  "date": "2025-09-18",
  "temperatureC": 40,
  "temperatureF": 103,
  "summary": "Scorching"
},
{
  "date": "2025-09-19",
  "temperatureC": 7,
  "temperatureF": 44,
  "summary": "Chilly"
}
]
```



⚠ Not Secure

48.216.149.220/health

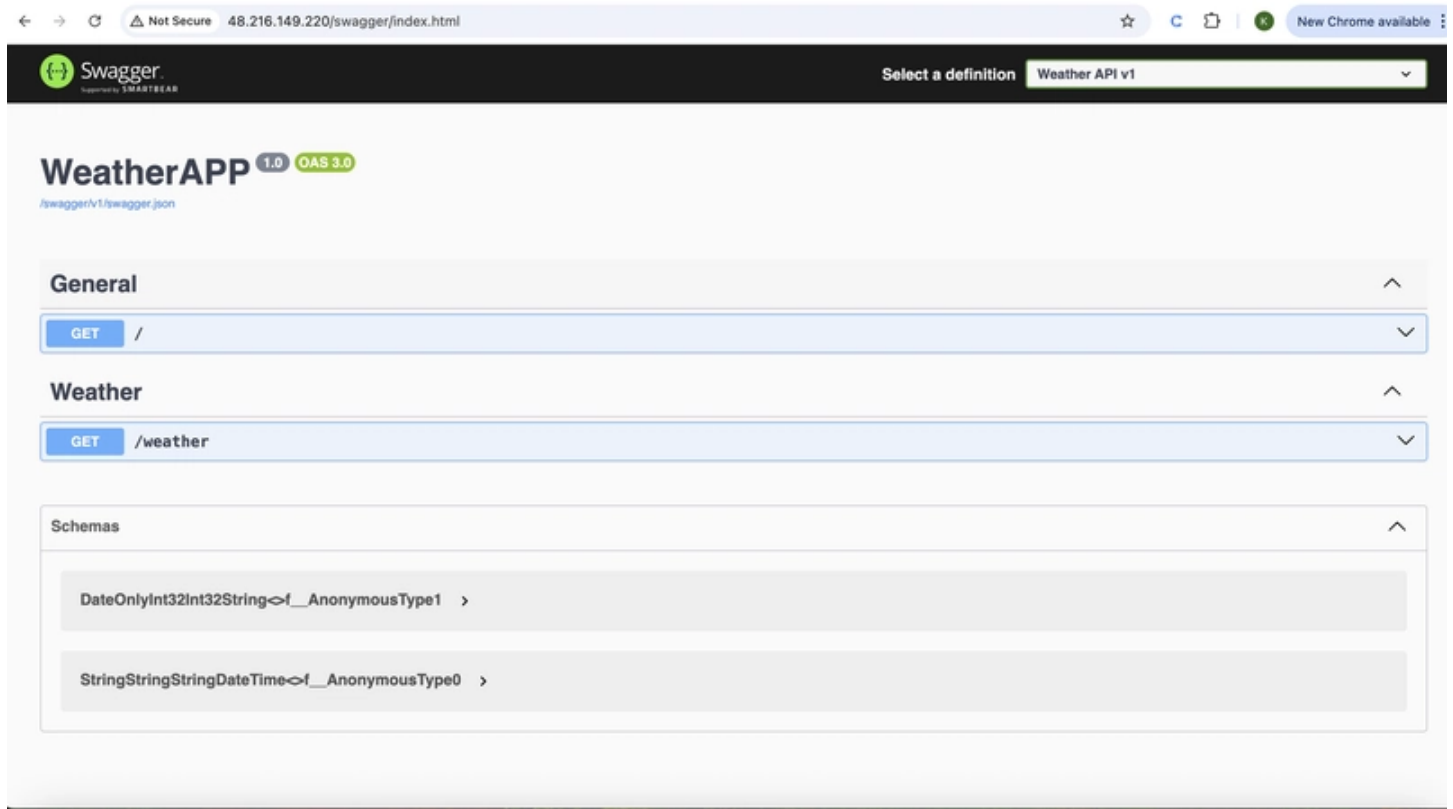
Healthy

Swagger UI (API docs):

open `http://YOUR-EXTERNAL-IP/swagger` ### macOS

start `http://YOUR-EXTERNAL-IP/swagger` ### Windows

In macOS



✅ Congratulations! Your .NET 8 Weather API is now fully containerized, deployed on Kubernetes, and publicly accessible through Azure AKS.

Step 7: Test Continuous Deployment

Now let's see GitHub Actions automatically deploy changes to AKS with no manual intervention required.

7.1 Make a Code Change

Edit `WeatherApp/Program.cs` to update the welcome message, for example:

```
app.MapGet("/", () => new
{
    Message = "Welcome to the Updated Weather App! 🌞",
    Version = "1.1.0",
    Environment = app.Environment.EnvironmentName,
```

```
    Timestamp = DateTime.UtcNow,  
    DeployedBy = "GitHub Actions"  
});
```

❏❏❏❏

This small change is perfect to test the pipeline.

7.2 Push the Changes

```
git add .  
git commit -m "Update welcome message and version"  
git push origin main
```

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS  AZURE  
● kosi@Mac WeatherApp % cd ..  
● kosi@Mac weather-app-demo % git add WeatherApp/Program.cs  
● kosi@Mac weather-app-demo % git commit -m "Update welcome message and version"  
[main d84d5ee] Update welcome message and version  
1 file changed, 49 insertions(+), 9 deletions(-)  
● kosi@Mac weather-app-demo % git push origin main  
Enumerating objects: 7, done.  
Counting objects: 100% (7/7), done.  
Delta compression using up to 10 threads  
Compressing objects: 100% (4/4), done.  
Writing objects: 100% (4/4), 1.23 KiB | 1.23 MiB/s, done.  
Total 4 (delta 1), reused 0 (delta 0), pack-reused 0 (from 0)  
remote: Resolving deltas: 100% (1/1), completed with 1 local object.  
To https://github.com/kosinachi/weather-app-demo.git  
76ce73b..d84d5ee  main -> main  
○ kosi@Mac weather-app-demo %
```

Pushing to the main branch triggers your GitHub Actions workflow automatically.

 kosinachi / weather-app-demo

Q Type  to search

 Code  Issues  Pull requests  **Actions**  Projects  Wiki  Security  Insights  Settings

[← Build and Deploy to AKS](#)

 **Update welcome message and version #7**

 **Summary**

Jobs
 build-and-deploy

Run details
-

Triggered via push 3 minutes ago

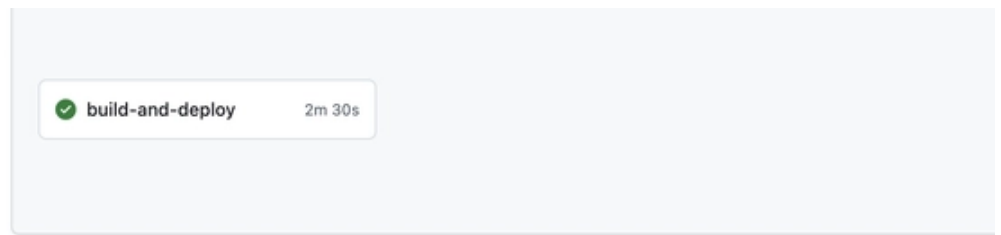
Status

Total duration

Artifacts

 kosinachi pushed  d84d5ee  main **Success** **2m 32s** -

deploy.yml
on: push



Open your GitHub repository → Actions tab.

Builds the .NET app

Builds & pushes the Docker image to ACR

Updates the Kubernetes deployment in AKS

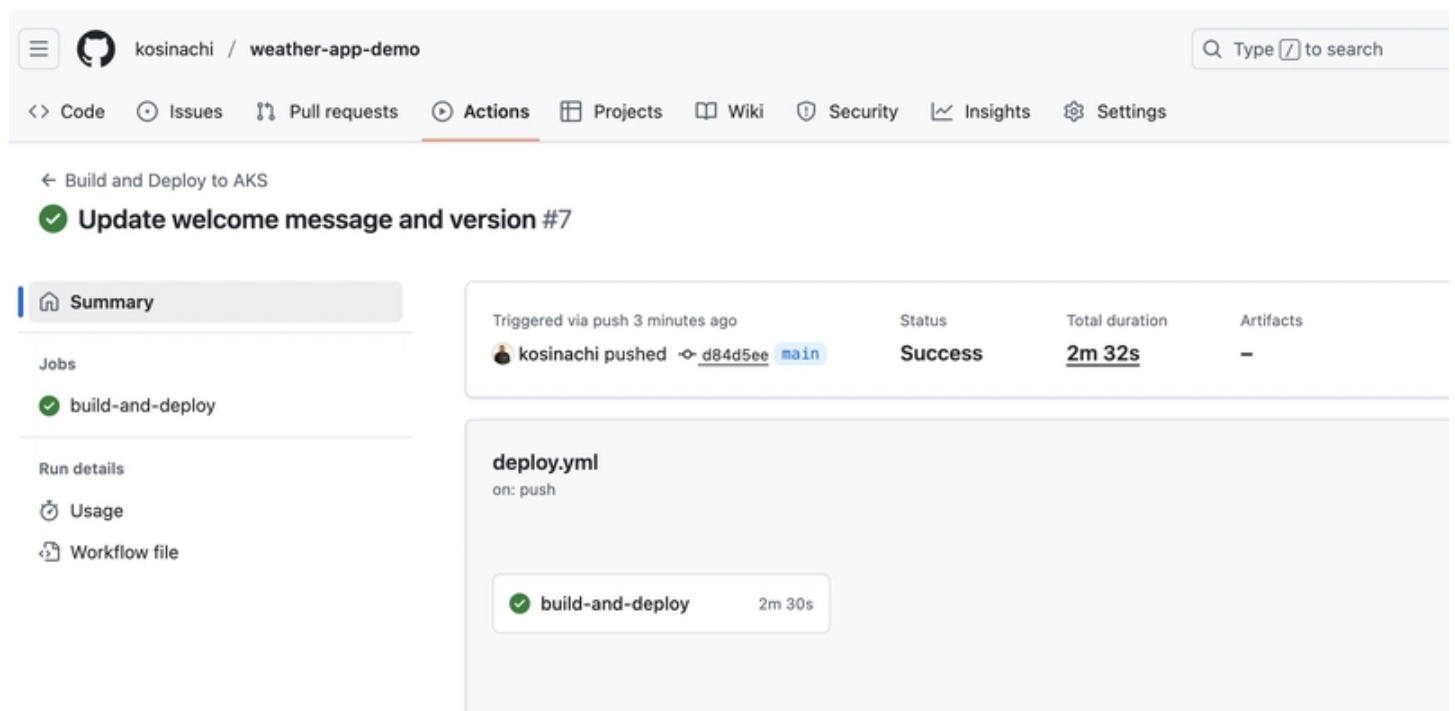
Once completed, test your updated app:

```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS AZURE
❯ zsh - weather-app-demo + ▢ 🗑️ ...
❯ kosi@Mac weather-app-demo % curl http://48.216.149.220/
{"message":"Welcome to the Updated Weather App! \u03C3\u03C4\u03C5","version":"1.1.0","environment":"Production","timestamp":"2025-09-14T17:49:57.643570Z","deployedBy":"GitHub Actions"}
❯ kosi@Mac weather-app-demo %

```

You should see the new welcome message and updated version, confirming that CI/CD is working flawlessly as confirmed below.



✅ Congratulations! we have now have a fully functional CI/CD pipeline:

Push code → automatically builds, pushes, and deploys

Changes reflected live in your AKS cluster

All without manually logging into Azure

Troubleshooting Guide

Even with the best setup, things can go wrong. Here are some common issues you might face and how to fix them.

Pods stuck in ImagePullBackOff or ErrImagePull

Symptoms:

Your pods won't start and show ImagePullBackOff.

Fixes:

```
# Check pod details
kubectl describe pod $(kubectl get pods -l app=weather-app -o
jsonpath='{.items[0].metadata.name}')

# Verify AKS can access your ACR
az aks check-acr \
  --resource-group student-demo \
  --name student-aks-cluster \
  --acr studentdemo2024acr.azurecr.io

# Re-attach ACR if needed
az aks update \
  --resource-group student-demo \
  --name student-aks-cluster \
  --attach-acr studentdemo2024acr

# Force pod recreation
kubectl delete pods -l app=weather-app
```

❏❏❏

2. kubectl: command not found

Symptoms:

kubectll isn't recognized.

Fixes:

```
# Check if installed
kubectll version --client
```

❌❌❌❌

If not installed, follow the official guide:

👉 Install kubectll

3. Docker daemon not running

Symptoms:

Docker commands fail with "Cannot connect to the Docker daemon."

Fixes:

Start Docker Desktop

Wait until the green status indicator shows

Restart your terminal

4. Access denied to Azure resources

Symptoms:

Azure CLI commands fail with "unauthorized" errors.

Fixes:

```
az logout
```

```
az login
```

```
az account show
```

Verify you are using the correct subscription.

5. GitHub Actions pipeline failing

Check these first:

All GitHub secrets are correctly set (AZURE_CREDENTIALS, ACR_NAME, etc.)

The Azure service principal has Contributor access to your resource group

Inspect the GitHub Actions logs for detailed error messages

6. Application not accessible via External IP

Fixes:

Check pods

```
kubectl get pods
```

Check service

```
kubectl get services
```

Inspect pod logs

```
kubectl logs deployment/weather-app
```

Describe the service

```
kubectl describe service weather-app-service
```

****Debugging Commands****

These are useful one-liners when things don't go as planned:

Cluster info

```
kubectl cluster-info
```

All resources in the namespace

```
kubectl get all
```

Pod logs

```
kubectl logs -l app=weather-app
```

Deployment details

```
kubectl describe deployment weather-app
```

Node status

```
kubectl get nodes -o wide
```

Monitor pods in real-time

```
kubectl get pods --watch
```

ACR repositories

```
az acr repository list --name studentdemo2024acr --output table
```

```
# ACR image tags
az acr repository show-tags \
  --name studentdemo2024acr \
  --repository weather-app \
  --output table
```

☐ ☐ ☐ ☐

✅ With these troubleshooting steps, you should be able to resolve most common deployment issues quickly.

Clean Up Resources

Once you are done experimenting, don't forget to clean up your Azure resources otherwise, you may keep paying for unused services.

You have two options:

Option 1: Delete Individual Resources

If you want to keep the resource group but remove specific resources:

Delete Kubernetes resources (Deployment + Service)

```
kubectl delete -f k8s/
```

Delete AKS cluster

```
az aks delete \
  --resource-group student-demo \
  --name student-aks-cluster \
  --yes
```

Delete Azure Container Registry (ACR)

```
az acr delete \
  --name studentdemo3121acr \
  --yes
```

Option 2: Delete Everything (Recommended)

If this was just a demo project, the easiest and safest way is to delete the entire resource group:

Delete the entire resource group (removes AKS, ACR, etc.)

```
az group delete \  
--name student-demo \  
--yes \  
--no-wait
```

⚠ Note: Your GitHub repository and workflow remain intact — only Azure resources will be deleted.

What we have Accomplished

By following along, we have gone from zero to cloud-native hero. Here's what we have built:

A Production-Ready .NET 8 Web API

RESTful endpoints with proper HTTP responses

Health checks for monitoring

Swagger documentation for easy API testing

Mastered Docker Containerization

Multi-stage Docker builds for optimization

Container security best practices

Local testing and validation

Deployed to Azure Kubernetes Service (AKS)

Infrastructure as Code with Azure CLI

Kubernetes manifests with proper resource requests/limits

LoadBalancer service for external access

Implemented CI/CD with GitHub Actions

Automated build and test pipeline

Container registry integration (ACR)

Zero-downtime deployments with rolling updates

Applied Cloud-Native Best Practices

Health, readiness, and startup probes for reliability

Resource limits for efficiency

Environment-specific configurations

Next Steps

Now that you have got the foundation, here's how you can level up:

Beginner Level

Add a Database: Integrate Azure SQL or PostgreSQL, use EF Core for persistence

Enhance Monitoring: Add Application Insights, set up log aggregation, track custom metrics

Implement Security: Add authentication with Azure AD, enable API versioning, and secure with HTTPS

Intermediate Level

Advanced Kubernetes Features: ConfigMaps & Secrets, Horizontal Pod Autoscaling, Ingress controllers with custom domains

Infrastructure as Code: Learn Azure Bicep or Terraform, adopt GitOps with ArgoCD, manage multiple environments

Observability Stack: Prometheus & Grafana for metrics, Jaeger for distributed tracing, ELK stack for centralized logging

You now have the skills to build, ship, and run modern .NET apps on the cloud like a pro. From here, the sky's the limit, scale it, secure it, and make it production-grade!

Azure Kubernetes Service (AKS) Deployment - Complete Guide Sheet

Whether you are deploying your first .NET app to Kubernetes or need a quick refresher, this cheat sheet gives you everything you need from Docker to AKS to GitHub Actions.

Core Concepts Explained

Kubernetes

Container Orchestrator: Manages your containers like a traffic controller

Scaling: Add/remove pods based on load

Self-Healing: Restarts crashed pods automatically

Load Balancing: Routes traffic evenly across replicas

Docker

Containerization: Package app + dependencies

Consistency: "Works on my machine" → "Works everywhere"

Isolation: Each container runs independently

 Azure Container Registry (ACR)

Private Docker Hub: Secure container storage

Version Control: Track different image versions

Integration: Works seamlessly with AKS

 Prerequisites

Tool Purpose Why You Need It

VS Code Code Editor Write and edit code

.NET 8 SDK Build Tool Compile your C# app

Docker Desktop Containers Build & test locally

Azure CLI Cloud Management Create/manage Azure resources

kubectl Kubernetes Client Deploy & manage apps on AKS

GitHub CLI Repo Management Create and push repos

Git Version Control Track code changes

Accounts Needed

GitHub → Free for repos & CI/CD

Azure → \$200 free credit to host apps

Azure Infrastructure

 Resource Group

```
az group create --name student-demo --location eastus`
```

❏ ❏ ❏ ❏

Resource group: Think of it as a project folder for all Azure resources

Container Registry (ACR)

```
az acr create --resource-group student-demo --name studentdemo2024acr --sku Basic
```

❏ ❏ ❏ ❏

Container registry is like an app store, where things can be stored, share, and download ready-to-run software packages called containers.

Think of it like your private Docker Hub

AKS Cluster

```
az aks create --resource-group student-demo --name student-aks-cluster --node-count 1
```

☐ ☐ ☐ ☐

This is like a managed Kubernetes cluster with auto-scaling & healing

.NET App Breakdown

🔑 Program.cs

Health Checks → `app.MapHealthChecks("/health");`

Swagger → API testing UI

Endpoints → `/`, `/weather`, `/health`

Kestrel Config

```
builder.WebHost.ConfigureKestrel(options => {  
    options.ListenAnyIP(8080);  
});
```

☐ ☐ ☐ ☐

Runs on port 8080 inside container

Docker Setup

📄 Dockerfile (multi-stage build)

Base → runtime image

Build → SDK to restore & publish

Final → small, production-ready image

.dockerignore

Ignore `bin/`, `obj/`, `.git/` → smaller, faster, more secure builds

Kubernetes Manifests

📄 deployment.yaml

`replicas: 2` → high availability

`resources.requests/limits` → efficient pod usage

health probes → auto-healing + traffic control

service.yaml


```
type: LoadBalancer
ports:
  - port: 80
    targetPort: 8080
```

❏ ❏ ❏ ❏

This exposes app with an external IP

GitHub Actions Workflow



CI/CD Pipeline Highlights

Checkout Code → actions/checkout@v4

Setup .NET → build/test app

Azure Login → via service principal

Build & Push Image → to ACR

Deploy to AKS → update deployment, restart rollout

Secrets Required

AZURE_CREDENTIALS → Service principal JSON

ACR_NAME → Your ACR name

RESOURCE_GROUP → Azure resource group

CLUSTER_NAME → AKS cluster name

Monitoring & Observability

Health Checks → HTTP 200 = healthy

Probes:

Liveness → restart if dead

Readiness → remove from traffic if unready

Startup → delay checks until app boots

Troubleshooting Essentials

Check Deployment → `kubectl rollout status deployment/weather-app`

Pod Logs → `kubectl logs <pod-name>`

Service Info → `kubectl describe service weather-app-service`

Image Issues → `az acr repository show-tags --name studentdemo2024acr --repository weather-app`

Quick Reference Commands

Docker

```
docker build -t app:tag .  
docker run -p 8080:8080 app:tag  
docker logs container-name
```

☐ ☐ ☐ ☐

Kubernetes

```
kubectl apply -f k8s/  
kubectl get pods  
kubectl logs -f <pod-name>  
kubectl scale deployment weather-app --replicas=3
```

☐ ☐ ☐ ☐

Azure CLI

```
az aks get-credentials --resource-group student-demo --name student-aks-cluster  
az acr login --name studentdemo2024acr  
az group delete --name student-demo --yes
```

☐ ☐ ☐ ☐

🔥 With this guide sheet which i call cheat sheet, you have all the key commands, configs, and concepts at your fingertips for deploying .NET apps on AKS with GitHub Actions.